

بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ

RISCV pipelined

topics

1. RISCV ISA

- 1.1 different extensions
- 1.2 RV32I Base Integer Instruction Set
- 1.3 different instructions

2. RISCV single-cycle implementation

- 2.1 different blocks explanation
- 2.2 Verilog code
- 2.3 simulation and code compilation
- 2.4 RTL elaboration and synthesis schematic
- 2.5 static-time analysis reports and utilization report
- 2.6 implementation schematic
- 2.7 static-time analysis reports and utilization report

3. RISCV pipeline implementation

- 3.1 different blocks explanation
- 3.2 Verilog code
- 3.3 simulation and code compilation
- 3.4 RTL elaboration and synthesis schematic
- 3.5 static-time analysis reports and utilization report
- 3.6 implementation schematic
- 3.7 static-time analysis reports and utilization report

4. REFERENCES

1. RISC-V ISA

1.1 different extensions

RISC-V has a bunch of different extensions in the ISA that are used for different purposes and need different hardware implementations.

One of them is the RV32I Base Integer Instruction Set, which is the base extension that should be in every RISC-V, and which will be the one we will make.

There are other extensions which are the:

1. "M" Extension for Integer Multiplication and Division
2. "A" Extension for Atomic Instructions
3. "F" Extension for Single-Precision Floating-Point
4. "D" Extension for Double-Precision Floating-Point
5. "Q" Extension for Quad-Precision Floating-Point
6. "C" Extension for Compressed Instructions
7. "B" Extension for Bit Manipulation
8. "J" Extension for Dynamically Translated Languages
9. "P" Extension for Packed-SIMD Instructions
10. "V" Standard Extension for Vector Operations
11. "Zifencei" Extension for Instruction-Fetch Fence

12. "Zicsr", Extension for Control and Status Register (CSR)

Instructions,

13. "Zicntr" and "Zihpm" Extensions for Counters,

14. "Zihintntl" Extension for Non-Temporal Locality Hints

And many more...

Some of these extensions are depending on the presence of another extension as examples:

"D" depends on "F" and "F" depends on "Zicsr".

Our focus now will be the RV32I Base Integer Instruction Set.

1.2 RV32I Base Integer Instruction Set

RV32I Base Integer Instruction Set is the main instruction set that should be in every RISC-V processor.

Each instruction is 32-bit width, and they are divided into some formats:

31	27	26	25	24	20	19	15	14	12	11		7	6		0	
funct7				rs2		rs1		funct3		rd			opcode		R-type	
imm[11:0]						rs1		funct3		rd			opcode		I-type	
imm[11:5]				rs2		rs1		funct3		imm[4:0]			opcode		S-type	
imm[12 10:5]				rs2		rs1		funct3		imm[4:1 11]			opcode		B-type	
imm[31:12]										rd			opcode		U-type	
imm[20 10:1 11 19:12]										rd			opcode		J-type	

Figure (1)

As we see in Figure (1), the base ISA is divided into six formats which are:

1. R-type format: used in the register instructions.
2. I-type format: used in the immediate (constants) instructions.
3. S-type format: used in the store instructions.
4. B-type format: used in the branch instructions.
5. U-type format: used in the “LUI” and the “AUIPC” instructions
6. J-type format: used in the jump instructions.

All these formats are divided into some blocks which are:

1. Opcode: which is used to define the type of the format or instruction.
2. Rd: which defines the register where the result will be saved in.
3. Funct3 & funct7: both are used to differentiate between the instructions that have the same opcode.
4. Rs1 & rs2: both are used to define the operands of the register operations.
5. Imm: is used to define the constant value that will be used in operations.

The imm value is divided into two blocks in “S” and “B” types to make the “funct3”, “rs1” and “rs2” blocks in the same bit range as the “R” format so the ISA format are going to be more standardized.

Also, the difference in the imm block between the “S” and “B” formats is that in “B” format the constant value is divided by 2 in the software rather than the hardware which was going to be shift operation to save resources and time as we will always shift the imm value in the “B” format with the same value to access the memory as the memory if word-aligned.

The same difference is between the “U” and the “J” formats.

- Note: we divided the value by 2 not 4 to make the same format compatible with the “C” extension where the instructions are only 16-bits.

1.3 Different instructions

RV32I Base Integer Instruction Set has bunch of instructions that we must implement in hardware to make a RISC-V processor.

RV32I Base Instruction Set						
imm[31:12]				rd	0110111	LUI
imm[31:12]				rd	0010111	AUIPC
imm[20 10:1 11 19:12]				rd	1101111	JAL
imm[11:0]		rs1	000	rd	1100111	JALR
imm[12 10:5]	rs2	rs1	000	imm[4:1 11]	1100011	BEQ
imm[12 10:5]	rs2	rs1	001	imm[4:1 11]	1100011	BNE
imm[12 10:5]	rs2	rs1	100	imm[4:1 11]	1100011	BLT
imm[12 10:5]	rs2	rs1	101	imm[4:1 11]	1100011	BGE
imm[12 10:5]	rs2	rs1	110	imm[4:1 11]	1100011	BLTU
imm[12 10:5]	rs2	rs1	111	imm[4:1 11]	1100011	BGEU
imm[11:0]		rs1	000	rd	0000011	LB
imm[11:0]		rs1	001	rd	0000011	LH
imm[11:0]		rs1	010	rd	0000011	LW
imm[11:0]		rs1	100	rd	0000011	LBU
imm[11:0]		rs1	101	rd	0000011	LHU
imm[11:5]	rs2	rs1	000	imm[4:0]	0100011	SB
imm[11:5]	rs2	rs1	001	imm[4:0]	0100011	SH
imm[11:5]	rs2	rs1	010	imm[4:0]	0100011	SW
imm[11:0]		rs1	000	rd	0010011	ADDI
imm[11:0]		rs1	010	rd	0010011	SLTI
imm[11:0]		rs1	011	rd	0010011	SLTIU
imm[11:0]		rs1	100	rd	0010011	XORI
imm[11:0]		rs1	110	rd	0010011	ORI
imm[11:0]		rs1	111	rd	0010011	ANDI
0000000	shamt	rs1	001	rd	0010011	SLLI
0000000	shamt	rs1	101	rd	0010011	SRLI
0100000	shamt	rs1	101	rd	0010011	SRAI
0000000	rs2	rs1	000	rd	0110011	ADD
0100000	rs2	rs1	000	rd	0110011	SUB
0000000	rs2	rs1	001	rd	0110011	SLL
0000000	rs2	rs1	010	rd	0110011	SLT
0000000	rs2	rs1	011	rd	0110011	SLTU
0000000	rs2	rs1	100	rd	0110011	XOR
	rs2	rs1	101	rd		0110011
	rs2	rs1	101	rd		0110011
	rs2	rs1	110	rd		0110011
	rs2	rs1	111	rd		0110011

Figure (2)

In figure (2), We can see all the instructions we need to implement as some of them are R-format like the “ADD” and the “SUB” instructions and other are I-format as the “ADDI” instructions and some are “J” instructions like the “JAL” instruction, and we can see that the instruction “JALR” is a jump instruction, but it uses the I-format.

2. RISCv single-cycle implementation

2.1 different blocks explanation

1- PC block.

The PC block is a counter that increment 4 to the input to get the new address of the next instruction which will be used as an input for the instruction memory.

2- instruction memory.

The instruction memory is a ROM that used to save the instructions so they can be read by the hardware

3- ALU

The ALU is used to make different arithmetical and logical operations

4- register file

The register file is a 32 registers used to save some data so they can be used in some operations.

5- data memory

RAM that is used to save data and it is slower than the register file but bigger

6- control unit

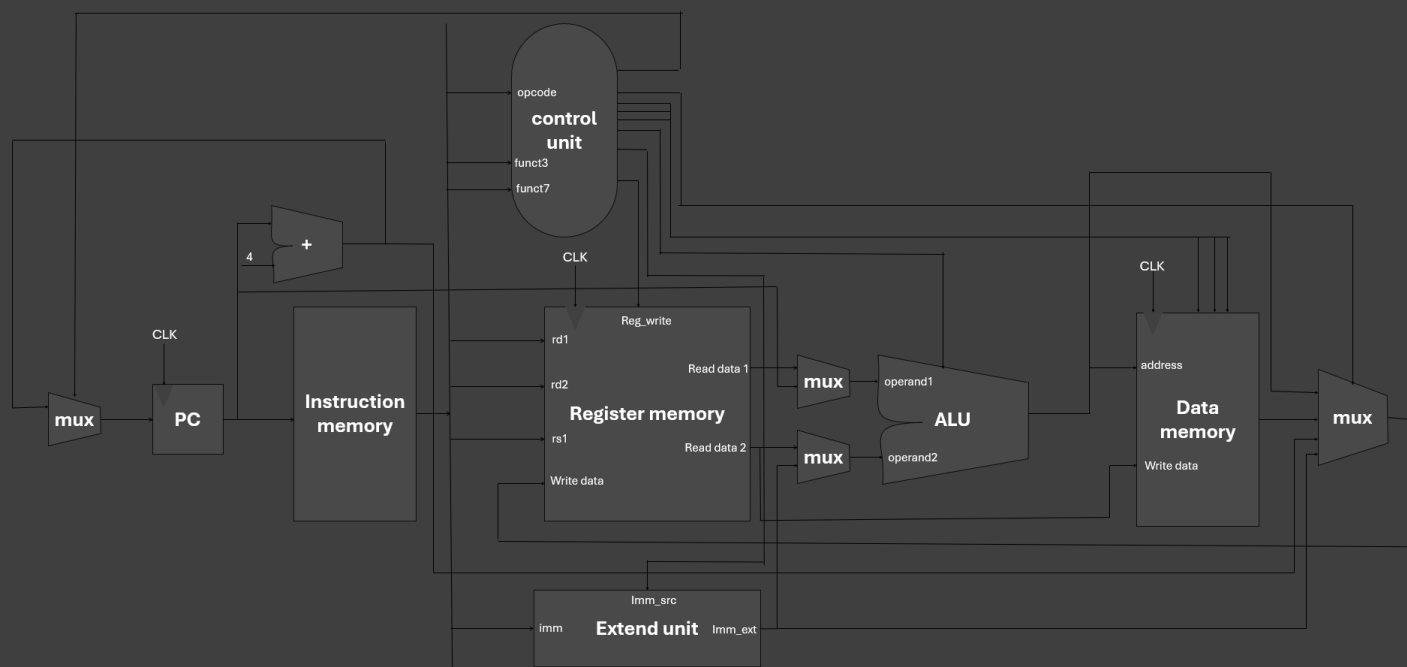
Used to take all the decisions in the processor and to control the flow of the data.

7- branch-jump decision block

Used to flag the processor if any branch or jump instructions are happening.

8- extend unit

Used to sign extend the immediates which are a part of the instructions where they are only 16 bits and needs to be extended to 32-bits as the other data.



2.2 Verilog code

PC

```
////////////////////////////////////////
/// Name: Abdelrahman Mohamed Ragab
/// module name: pc
////////////////////////////////////////

module pc (clk, rst, in, out);

    // Base address for the first instruction
    parameter BASE_INSTRUCTION = 32'h00000000;
    // Size of the instruction (32 bits)
    parameter SIZE = 32;

    // input port
    input [SIZE-1:0] in;           // Input address (32 bits)
    input clk, rst;               // Clock and reset signals

    // output port
    output reg [SIZE-1:0] out;     // Output address (32 bits)

    // Always block triggered on the rising edge of the clock
    always @(posedge clk) begin
        if (rst) begin
            // If reset is high, set the output to the base instruction address
            out <= BASE_INSTRUCTION;
        end else begin
            // Otherwise, increment the input address by 4 and assign it to the output
            out <= in;
        end
    end
end

endmodule
```

Instruction memory

```
////////////////////////////////////
// Name: Abdelrahman Mohamed Ragab
// module name: imem
////////////////////////////////////

module imem (addr, instruction);

    // Size of the address and instruction (32 bits)
    parameter SIZE = 32;
    // Base address for the instruction memory
    parameter BASE_ADDRESS = 32'h00000000;
    // Size of the instruction memory (252 MB)
    parameter mem_SIZE = 2000;                                //252 * 2**20;

    // input port
    input [SIZE-1:0] addr;                                     // Input address (32 bits)

    // output port
    output [SIZE-1:0] instruction;                             // Output instruction (32 bits)

    // Memory array (8-bit wide, mem_SIZE entries)
    reg [31:0] mem [0:mem_SIZE-1];

    // Initial block to load the instruction memory from a file
    initial begin
        // Load the memory contents from the file "imem.mem"
        $readmemh("imem.mem", mem);
    end

    // Assign the instruction output to the value stored at the specified address
    assign instruction = mem[addr - BASE_ADDRESS];

endmodule
```

ALU

```
////////////////////////////////////
// Name: Abdelrahman Mohamed Ragab
// module name: ALU
////////////////////////////////////

module ALU(operand_1, operand_2, result, ALU_control);

    parameter SIZE = 32;

    // input port
    input signed [SIZE-1:0] operand_1, operand_2;             // Input operands (32 bits each)
    input [4:0] ALU_control;                                   // ALU control signal (5 bits)

    // output port
    output reg signed [SIZE-1:0] result;                       // ALU result (32 bits)

    // Always block to perform ALU operations based on ALU_control
    always @(*) begin
        case (ALU_control)
            5'b00000: result = operand_1 + operand_2; // ADD
            5'b00010: result = operand_1 - operand_2; // SUB
            5'b00100: result = operand_1 << operand_2; // SLL
            5'b01000: result = operand_1 < operand_2 ? 1 : 0; // SLT
            5'b01100: result = $unsigned(operand_1) < $unsigned(operand_2) ? 1 : 0; // SLTU
            5'b10000: result = operand_1 ^ operand_2; // XOR
            5'b10100: result = operand_1 >> operand_2; // SRL
            5'b10110: result = operand_1 >>> operand_2; // SRA
            5'b11000: result = operand_1 | operand_2; // OR
            5'b11100: result = operand_1 & operand_2; // AND
            default: result = 0; // Default case
        endcase
    end

endmodule
```

Extend unit

```
////////////////////////////////////
/// Name: Abdelrahman Mohamed Ragab
/// module name: imm_extend
////////////////////////////////////

module imm_extend(imm, extended_imm, imm_src);

    parameter SIZE = 32;

    // input port
    input [31:7] imm; // Input immediate value (25 bits)
    input [2:0] imm_src; // Input source type (3 bits)

    // output port
    output reg [SIZE-1:0] extended_imm; // Output extended immediate value (32 bits)

    // Select the immediate extension based on the instruction format
    always @(*) begin
        case (imm_src)
            // U-FORMAT-INSTRUCTIONS
            0: begin
                extended_imm = {imm[31:12], 12'b0};
            end
            // J-FORMAT-INSTRUCTIONS
            1: begin
                extended_imm = {{12{imm[31]}}, imm[19:12], imm[20], imm[30:21], 1'b0};
            end
            // S-FORMAT-INSTRUCTIONS
            2: begin
                extended_imm = {{20{imm[31]}}, imm[31:25], imm[11:7]};
            end
            // B-FORMAT-INSTRUCTIONS
            3: begin
                extended_imm = {{20{imm[31]}}, imm[7], imm[30:25], imm[11:8], 1'b0};
            end
            // I-FORMAT-SIGNED-INSTRUCTIONS & JALR instruction
            4: begin
                extended_imm = {{20{imm[31]}}, imm[31:20]};
            end
            // I-FORMAT-SHIFT-INSTRUCTIONS
            5: begin
                extended_imm = {{27{imm[31]}}, imm[24:20]};
            end
            // I-FORMAT-UNSIGNED-INSTRUCTIONS
            6: begin
                extended_imm = {20'b0, imm[31:20]};
            end
            // Default case for invalid instruction format
            default: extended_imm = 32'hxxxxxxxx;
        endcase
    end
endmodule
```

Branch_jump decision unit

```
////////////////////////////////////
/// Name: Abdelrahman Mohamed Ragab
/// module name: jump_branch_decision
////////////////////////////////////

module jump_branch_decision(operand_1, operand_2, zero, neg, less_than, jump_branch, pc_src);

    parameter SIZE = 32;

    // input port
    input [SIZE-1:0] operand_1, operand_2; // Input operands (32 bits each)
    input [2:0] jump_branch; // Jump or branch signal (3 bits)
    input zero, neg, less_than; // Zero and negative and less than flags

    // output port
    output reg pc_src; // Output PC source (1 bits)

    // Always block to decide the PC source
    always @(*) begin
        if (jump_branch == 3'b010) begin
            pc_src <= 0; // PC source is the incremented PC
        end else if (jump_branch == 3'b011) begin
            pc_src <= 1; // PC source is the jump target address
        end else begin
            case (jump_branch)
                3'b000: pc_src <= (zero) ? 1 : 0; // BEQ
                3'b001: pc_src <= (!zero) ? 1 : 0; // BNE
                3'b100: pc_src <= (neg) ? 1 : 0; // BLT
                3'b101: pc_src <= (!neg || zero) ? 1 : 0; // BGE
                3'b110: pc_src <= (less_than) ? 1 : 0; // BLTU
                3'b111: pc_src <= (!less_than || zero) ? 1 : 0; // BGEU
                default: pc_src <= 0; // Default case
            endcase
        end
    end

    // Zero flag is high if result is zero
    assign zero = (operand_1 - operand_2 == 0);

    // Negative flag is high if result is negative
    assign neg = (operand_1 < operand_2);

    // UNSIGNED LESS THAN FLAG
    assign less_than = ($unsigned(operand_1) < $unsigned(operand_2));

endmodule
```

Register file

```
////////////////////////////////////
// Name: Abdelrahman Mohamed Ragab
// module name: reg_file
////////////////////////////////////

module reg_file (read_address_1, read_address_2, write_address, write_data, clk, write_enable, read_data_1, read_data_2, rst);

    // Size of the register file (32 registers, each 32 bits wide)
    parameter SIZE = 32;

    // input port
    input [SIZE-1:0] write_data;
    input [SIZE-1:0] read_address_1, read_address_2, write_address;
    input clk, write_enable, rst;

    // output port
    output [SIZE-1:0] read_data_1, read_data_2;

    // Register file array (32 registers, each 32 bits wide)
    reg [SIZE-1:0] reg_file [0:SIZE-1];

    // Integer for loop iteration
    integer i;

    // Always block triggered on the rising edge of the clock
    always @(posedge clk) begin
        if (rst) begin
            // If reset is high, initialize all registers to 0
            for (i = 0; i < SIZE; i = i + 1) begin
                reg_file[i] <= 0;
            end
            reg_file[2] <= 32'h00000500;
        end else if (write_enable) begin
            // If write enable is high, write data to the specified register
            if (!write_address) begin
                reg_file[write_address] <= 0;
            end else begin
                reg_file[write_address] <= write_data;
            end
        end
    end

    // Assign the read data outputs to the values stored in the specified registers
    assign read_data_1 = reg_file[read_address_1];
    assign read_data_2 = reg_file[read_address_2];

endmodule
```

CONTROL UNIT

```
////////////////////////////////////
// Name: Abdelrahman Mohamed Ragab
// Module Name: control_unit
////////////////////////////////////

module control_unit(op, funct3, funct7, jump_branch, result_src, mem_write, alu_control, alu_src, imm_src, reg_write, data_size, extension_type);

    // input port
    input [6:0] op;
    input [6:0] funct7;
    input [2:0] funct3;

    // output port
    output reg [4:0] alu_control;
    output reg [2:0] imm_src;
    output reg [1:0] result_src;
    output reg [1:0] alu_src;
    output reg [2:0] jump_branch;
    output reg [1:0] data_size;
    output reg extension_type;
    output reg mem_write;
    output reg reg_write;

    // Always block to determine the control signals based on the opcode and funct3 and funct7
    always @(*) begin
        //alu_control
        case (op)
            // R-FORMAT-INSTRUCTIONS
            7'b0110011: case ({funct7, funct3})
                10'b00000000: alu_control <= 5'b000000; // ADD
                10'b01000000: alu_control <= 5'b000010; // SUB
                10'b00000001: alu_control <= 5'b000100; // SLL
                10'b00000010: alu_control <= 5'b010000; // SLT
                10'b00000011: alu_control <= 5'b011000; // SLTU
                10'b00000100: alu_control <= 5'b100000; // XOR
                10'b00000101: alu_control <= 5'b101000; // SRL
                10'b01000101: alu_control <= 5'b101100; // SRA
                10'b00000110: alu_control <= 5'b110000; // OR
                10'b00000111: alu_control <= 5'b111000; // AND
                default: alu_control <= 5'b000000; // Default case
            endcase
            // I-FORMAT-INSTRUCTIONS
            7'b0010011: case (funct3)
                3'b000: alu_control <= 5'b000000; // ADDI
                3'b001: alu_control <= 5'b001000; // SLLI
                3'b010: alu_control <= 5'b010000; // SLTI
                3'b011: alu_control <= 5'b011000; // SLTIU
                3'b100: alu_control <= 5'b100000; // XORI
                3'b110: alu_control <= 5'b110000; // ORI
                3'b111: alu_control <= 5'b111000; // ANDI
                3'b101: case (funct7)
                    7'b0000000: alu_control <= 5'b101000; // SRLI
                    7'b0100000: alu_control <= 5'b101010; // SRAI
                    default: alu_control <= 5'b000000; // Default case
                endcase
            endcase
            default: alu_control <= 5'b000000; // Default case
        endcase
    end
```

```

// LOAD-INSTRUCTIONS
7'b0000011: alu_control = 5'b00000; // ADD
// S-FORMAT-INSTRUCTIONS
7'b0100011: alu_control = 5'b00000; // ADD
// J-FORMAT-INSTRUCTIONS
7'b1101111, 7'b1100111: alu_control <= 5'b00000; // ADD
// U-FORMAT-INSTRUCTIONS
7'b0010111, 7'b0110111: alu_control = 5'b00000; // ADD
// B-FORMAT-INSTRUCTIONS
7'b1100011: alu_control <= 5'b00000; // ADD
// PAUSE INSTRUCTION
7'b0001111: alu_control = 5'b00000; // ADD
default: alu_control = 4'b0000;
endcase

//imm_src
case (op)
// 1-FORMAT-SIGNED-INSTRUCTIONS & JALR instruction
7'b0010011, 7'b0000011, 7'b1100111: imm_src <= 3'b100;
// S-FORMAT-INSTRUCTIONS
7'b0100011: imm_src <= 3'b010;
// B-FORMAT-INSTRUCTIONS
7'b1100011: imm_src <= 3'b011;
// U-FORMAT-INSTRUCTIONS
7'b0110111, 7'b0010111: imm_src <= 3'b000;
// JAL-INSTRUCTION
7'b1101111: imm_src <= 3'b001;
// I-FORMAT-SHIFT-INSTRUCTIONS
7'b0010011: imm_src <= 3'b101;
// I-FORMAT-UNSIGNED-INSTRUCTIONS
7'b0010011: imm_src <= 3'b110;
default: imm_src <= 3'b000;
endcase

//result_src
case (op)
// 1-FORMAT-ARITHMATIC-INSTRUCTIONS & R-FORMAT-INSTRUCTIONS & AUIPC-INSTRUCTION
7'b0010011, 7'b0110011, 7'b0010111: result_src <= 2'b00;
// I-FORMAT-LOAD-INSTRUCTIONS
7'b0000011: result_src <= 2'b01;
// JAL-INSTRUCTION & JALR-INSTRUCTION
7'b1101111, 7'b1100111: result_src <= 2'b10;
// LUI-INSTRUCTION
7'b0110111: result_src <= 2'b11;
default: result_src <= 2'b00;
endcase

//alu_src
case (op)
// R-FORMAT-INSTRUCTIONS
7'b0110011: alu_src <= 2'b00;
// I-FORMAT-INSTRUCTIONS & S-FORMAT-INSTRUCTIONS & //JALR-INSTRUCTION
7'b0000011, 7'b0010011, 7'b0100011, 7'b1100111: alu_src <= 2'b01;
// B-FORMAT-INSTRUCTIONS & JAL-INSTRUCTION & AUIPC-INSTRUCTION
7'b1101111, 7'b1100011, 7'b0010111: alu_src <= 2'b11;
default: alu_src <= 2'b00;
endcase

```

```

//jump_branch
case (op)
// JAL-INSTRUCTION & JALR-INSTRUCTION
7'b1101111, 7'b1100111: jump_branch <= 3'b011; // 3
// B-FORMAT-INSTRUCTIONS
7'b1100011: case (func3)
3'b000: jump_branch <= 3'b000; // BEQ
3'b001: jump_branch <= 3'b001; // BNE
3'b100: jump_branch <= 3'b100; // BLT
3'b101: jump_branch <= 3'b101; // BGE
3'b110: jump_branch <= 3'b110; // BLTU
3'b111: jump_branch <= 3'b111; // BGEU
default: jump_branch <= 3'b010; // NO
endcase
default: jump_branch <= 3'b010; // NO
endcase

//data_size
case (op)
7'b0000011: case (func3)
3'b000: data_size <= 2'b00; // LB
3'b001: data_size <= 2'b01; // LH
3'b100: data_size <= 2'b10; // LW
3'b101: data_size <= 2'b00; // LBU
3'b110: data_size <= 2'b01; // LHU
default: data_size <= 2'b00; // Default case
endcase
7'b0100011: case (func3)
3'b000: data_size <= 2'b00; // SB
3'b001: data_size <= 2'b01; // SH
3'b010: data_size <= 2'b10; // SW
endcase
default: data_size <= 2'b00; // Default case
endcase

//extension_type
case (op)
7'b0000011: case (func3)
3'b000, 3'b001: extension_type <= 1'b0; // LB, LH
3'b100, 3'b101: extension_type <= 1'b1; // LW, LHU
default: extension_type <= 1'b0; // Default case
endcase
default: extension_type <= 1'b0; // Default case
endcase

//mem_write
case (op)
// S-FORMAT-INSTRUCTIONS
7'b0100011: mem_write <= 1'b1;
default: mem_write <= 1'b0;
endcase

//reg_write
case (op)
// 1-FORMAT-INSTRUCTIONS & R-FORMAT-INSTRUCTIONS & JAL-INSTRUCTION & JALR-INSTRUCTION & AUIPC-INSTRUCTION
7'b0000011, 7'b0010011, 7'b0110011, 7'b1101111, 7'b1100111, 7'b0010111: reg_write <= 1'b1;
default: reg_write <= 1'b0;
endcase
end
endmodule

```

Data memory

```

////////////////////////////////////
// Name: Abdelrahman Mohamed Ragab
// Module-Name: data_memory
////////////////////////////////////

module data_memory(address, write_data, read_data, clk, rst, data_size, extension_type, write_enable);

    // Size of the address and instruction (32 bits)
    parameter SIZE = 32;
    // Base address for the instruction memory
    parameter BASE_ADDRESS = 32'h00000000;
    // Size of the instruction memory (1792 MB)
    parameter mem_SIZE = 2000; //1792 * 2**20;

    // Input port
    input [SIZE-1:0] write_data; // Input data to be written to the data memory
    input [SIZE-1:0] address; // Address for reading and writing data
    input [1:0] data_size; // Data size (2 bits)
    input clk, rst, write_enable, extension_type; // Clock, write enable, extension_type and reset signals

    // output port
    output reg [SIZE-1:0] read_data; // Data read from the specified address

    // Memory array (8-bit wide)
    reg [31:0] data_memory [0:mem_SIZE-1];

    // Integer for loop iteration
    integer i;

    // reg byte_enable;
    reg [3:0] byte_enable;

    // Always block triggered on the rising edge of the clock
    always @(*) begin
        case (address[1:0])
            0: byte_enable = 4'b0001;
            1: byte_enable = 4'b0010;
            2: byte_enable = 4'b0100;
            3: byte_enable = 4'b1000;
            default: byte_enable = 4'b0000;
        endcase
    end

    // Always block triggered on the rising edge of the clock
    always @(posedge clk) begin
        if (write_enable) begin
            // If write enable is high, write data to the specified address
            case (data_size)
                2'b00: begin
                    // Store byte
                    if (byte_enable[0]) data_memory[address - BASE_ADDRESS][7:0] <= write_data[7:0];
                    if (byte_enable[1]) data_memory[address - BASE_ADDRESS][15:8] <= write_data[7:0];
                    if (byte_enable[2]) data_memory[address - BASE_ADDRESS][23:16] <= write_data[7:0];
                    if (byte_enable[3]) data_memory[address - BASE_ADDRESS][31:24] <= write_data[7:0];
                end
                2'b01: begin
                    // Store halfword
                    if (byte_enable[0]) data_memory[address - BASE_ADDRESS][15:0] <= write_data[15:0];
                    if (byte_enable[2]) data_memory[address - BASE_ADDRESS][31:16] <= write_data[15:0];
                end
                2'b10: begin
                    // Store word
                    data_memory[address - BASE_ADDRESS] <= write_data[31:0];
                end
                default: begin
                    // Default case: store zero
                    data_memory[address - BASE_ADDRESS] <= 8'h00;
                end
            endcase
        end
    end
end

```

```

always @(*) begin
    case (data_size)
        2'b00: begin
            if (!extension_type) begin
                // Byte-Sign Extend
                if (byte_enable[0]) read_data <= {24{data_memory[address - BASE_ADDRESS][7]}}, data_memory[address - BASE_ADDRESS][7:0];
                if (byte_enable[1]) read_data <= {24{data_memory[address - BASE_ADDRESS][15]}}, data_memory[address - BASE_ADDRESS][15:8];
                if (byte_enable[2]) read_data <= {24{data_memory[address - BASE_ADDRESS][23]}}, data_memory[address - BASE_ADDRESS][23:16];
                if (byte_enable[3]) read_data <= {24{data_memory[address - BASE_ADDRESS][31]}}, data_memory[address - BASE_ADDRESS][31:24];
            end else begin
                // Byte-UNSign Extend
                if (byte_enable[0]) read_data <= {24'h000000, data_memory[address - BASE_ADDRESS][7:0]};
                if (byte_enable[1]) read_data <= {24'h000000, data_memory[address - BASE_ADDRESS][15:8]};
                if (byte_enable[2]) read_data <= {24'h000000, data_memory[address - BASE_ADDRESS][23:16]};
                if (byte_enable[3]) read_data <= {24'h000000, data_memory[address - BASE_ADDRESS][31:24]};
            end
        end
        2'b01: begin
            if (!extension_type) begin
                // Halfword-Sign Extend
                if (byte_enable[0]) read_data <= {16{data_memory[address - BASE_ADDRESS][15]}}, data_memory[address - BASE_ADDRESS][15:0];
                if (byte_enable[2]) read_data <= {16{data_memory[address - BASE_ADDRESS][31]}}, data_memory[address - BASE_ADDRESS][31:16];
            end else begin
                // Halfword-Zero Extend
                if (byte_enable[0]) read_data <= {16'h0000, data_memory[address - BASE_ADDRESS][15:0]};
                if (byte_enable[2]) read_data <= {16'h0000, data_memory[address - BASE_ADDRESS][31:16]};
            end
        end
        2'b10: read_data <= data_memory[address - BASE_ADDRESS]; // Word
        default: read_data <= 0;
    endcase
end
endmodule

```

top

```
////////////////////////////////////////
/// Name: Abdelrahman Mohamed Ragab
/// Module-Name: top
////////////////////////////////////////

module top(clk, rst, alu_result);

    // input port
    input clk;                // Clock signal
    input rst;                // reset signal

    // wire declaration
    wire [31:0] instruction;    // Instruction read from the instruction memory
    wire [31:0] read_data_1, read_data_2; // Data read from the register file
    wire [31:0] read_data;      // Data read from the data memory
    wire [31:0] write_data;     // Data to be written to the data memory
    wire [31:0] in;             // Input address from the program counter
    wire [31:0] out;            // Output address from the program counter
    wire [31:0] extended_imm;   // Extended immediate value
    output [31:0] alu_result;    // ALU result
    wire [31:0] operand_1;      // ALU operand 1
    wire [31:0] operand_2;      // ALU operand 2
    wire [4:0] alu_control;     // ALU control signal
    wire [2:0] jump_branch;     // Jump or branch signal
    wire [2:0] imm_src;         // Immediate source
    wire [1:0] result_src, alu_src; // Result source and ALU source
    wire [1:0] data_size;       // Data size
    wire pc_src, mem_write, reg_write; // Control signals for program counter, data memory, and register file
    wire extension_type;        // Extension type
    wire zero, neg, less_than;  // Zero, negative, and less than flags

    // module instantiation

    // Mux to select the next PC value
    mux2 mux2_pc_value_inst(.operand_1(out + 4), .operand_2(alu_result), .result(in), .select(pc_src));

    // Program Counter
    pc pc_inst(.clk(clk), .rst(rst), .in(in), .out(out));

    // Instruction Memory
    imm imm_inst(.addr((2'b00,out[31:2])), .instruction(instruction));

    // Mux to select the register input value
    mux4 mux4_register_input_inst(.operand_1(alu_result), .operand_2(read_data), .operand_3(out + 4), .operand_4(extended_imm), .result(write_data), .select(result_src));

    // Register File
    reg_file reg_file_inst(.read_address_1(instruction[19:15]), .read_address_2(instruction[24:20]), .write_address(instruction[11:7]), .write_data(write_data), .clk(clk), .write_enable(mem_write), .read_data_1(read_data_1), .read_data_2(read_data_2), .rst(rst));

    // Mux to select the first operand for the ALU
    mux2 mux2_ADDER_OPERAND_1_inst(.operand_1(read_data_1), .operand_2(out), .result(operand_1), .select(alu_src[1]));

    // Mux to select the second operand for the ALU
    mux2 mux2_ADDER_OPERAND_2_inst(.operand_1(read_data_2), .operand_2(extended_imm), .result(operand_2), .select(alu_src[0]));

    // ALU
    ALU alu_inst(.operand_1(operand_1), .operand_2(operand_2), .result(alu_result), .ALU_control(alu_control));

    // Data Memory
    data_memory data_memory_inst(.address((2'b00,alu_result[31:2])), .write_data(read_data_2), .read_data(read_data), .clk(clk), .rst(rst), .data_size(data_size), .extension_type(extension_type), .write_enable(mem_write));

    // Immediate Value Extender
    imm_extend imm_extend_inst(.imm(instruction[31:7]), .extended_imm(extended_imm), .imm_src(imm_src));

    // Control Unit
    control_unit control_unit_inst(.op(instruction[6:0]), .funct3(instruction[14:12]), .funct7(instruction[31:25]), .jump_branch(jump_branch), .result_src(result_src), .mem_write(mem_write), .alu_control(alu_control), .alu_src(alu_src), .imm_src(imm_src), .reg_write(reg_write), .data_size(data_size), .extension_type(extension_type));

    // Jump and Branch Decision Unit
    jump_branch_decision jump_branch_decision_inst(.operand_1(read_data_1), .operand_2(read_data_2), .zero(zero), .neg(neg), .less_than(less_than), .jump_branch(jump_branch), .pc_src(pc_src));

endmodule
```

The above code represents the single-cycle RISCv processor with all the base integer instructions signed and unsigned with the same functions mentioned above for each block.

2.3 simulation and code compilation

For verification the code functionality I used to different codes which are:

The first code is a code in the DDCA reference by “Sarah L Harris” and “David Money Harris”.

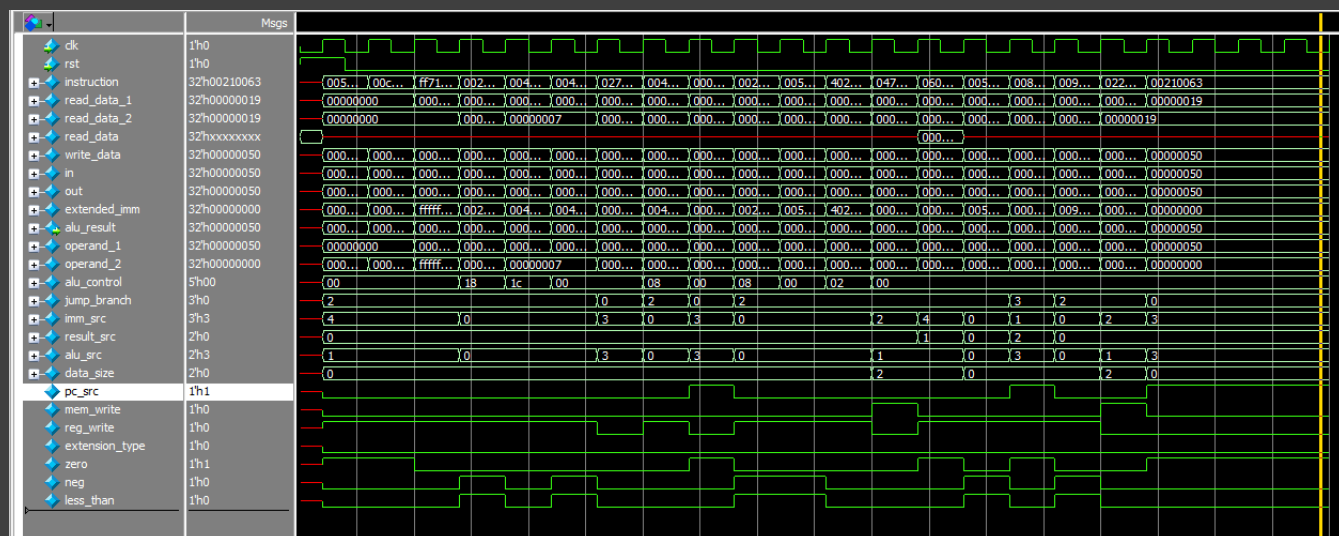
The second code was a C-code which was compiled and linked with the hardware using a linker script then used inside the RISCV in form of machine code.

The first code:

#	RISC-V Assembly	Description	Address	Machine Code
main:	addi x2, x0, 5	# x2 = 5	0	00500113
	addi x3, x0, 12	# x3 = 12	4	00C00193
	addi x7, x3, -9	# x7 = (12 - 9) = 3	8	FF718393
	or x4, x7, x2	# x4 = (3 OR 5) = 7	C	0023E233
	and x5, x3, x4	# x5 = (12 AND 7) = 4	10	0041F2B3
	add x5, x5, x4	# x5 = 4 + 7 = 11	14	004282B3
	beq x5, x7, end	# shouldn't be taken	18	02728863
	slt x4, x3, x4	# x4 = (12 < 7) = 0	1C	0041A233
	beq x4, x0, around	# should be taken	20	00020463
	addi x5, x0, 0	# shouldn't execute	24	00000293
around:	slt x4, x7, x2	# x4 = (3 < 5) = 1	28	0023A233
	add x7, x4, x5	# x7 = (1 + 11) = 12	2C	005203B3
	sub x7, x7, x2	# x7 = (12 - 5) = 7	30	402383B3
	sw x7, 84(x3)	# [96] = 7	34	0471AA23
	lw x2, 96(x0)	# x2 = [96] = 7	38	06002103
	add x9, x2, x5	# x9 = (7 + 11) = 18	3C	005104B3
	jal x3, end	# jump to end, x3 = 0x44	40	008001EF
	addi x2, x0, 1	# shouldn't execute	44	00100113
end:	add x2, x2, x9	# x2 = (7 + 18) = 25	48	00910133
	sw x2, 0x20(x3)	# [100] = 25	4C	0221A023
done:	beq x2, x2, done	# infinite loop	50	00210063

Figure 7.64 riscvtest.s

The simulation:



Register file contents:

[illegible]

Data memory contents:

00000000	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX
00000014	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	00000007	00000019
00000028	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX
0000003c	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX

As we can see the values inside the data memory and the register file is the same as the values written in the reference.

The second code:

To get the machine code from the C-code we need to do some steps first:

1- download the toolchain for RISCV that contain the compiler we will use.

2- compile the C-code

```
riscv-none-elf-gcc -march=rv32i -mabi=ilp32 -mno-relax -ffreestanding -nostdlib --std=c99 -c s.c -o output.o
```

“riscv-none-elf-gcc -march=rv32i -mabi=ilp32 -mno-relax -ffreestanding -nostdlib --std=c99 -c s.c -o output.o”

By using this command line in the cmd we would compile the C-code in s.c file.

riscv-none-elf-gcc: this is the name of the compiler.

-march=rv32i, -mabi=ilp32: these flags identify the type of architecture we use and the extensions.

-ffreestanding, -nostdlib, --std=c99: all these flags are for the C-code do identify the libraries.

-c: this flag is for compiling.

s.c is the C-code file.

Output.o is the output object file.

3- write the linker script

```
MEMORY
{
    ROM : ORIGIN = 0x00000000, LENGTH = 0x000007D0
    RAM : ORIGIN = 0x00000000, LENGTH = 0x000007D0
}

SECTIONS
{
    .text.main :
    {
        . = 0x00000000;
        *(.text);
        end = .;
        . = ALIGN(4);
    } > ROM

    .text :
    {
        *(.text);
    } > ROM

    .data :
    {
        *(.data)
        *(.bss)
    } > RAM
}
```

This is the linker script that identify the offset of the memory and their sizes.

It also determine which memory the code and the data will be stored into and in what order.

4- link the object file

```
>riscv-none-elf-gcc -march=rv32i -mabi=ilp32 -mno-relax -ffreestanding -nostdlib --std=c99 -T linker.lds -Wl,-Map,output.map output.o -o output.elf
```

“riscv-none-elf-gcc -march=rv32i -mabi=ilp32 -mno-relax -ffreestanding -nostdlib --std=c99 -T linker.lds -Wl,-Map,output.map output.o -o output.elf”

This line code link the object file to the hardware and generate an elf file.

-Wl: this flag used to make us able to write a bunch of flags after each other with a comma to separate them

-Map,output.map: this part is used to generate a map file.

-T: this flag is for linking.

linker.lds: the name of the linker script.

The map file used to define the Memory map and the place of each Section in the memory.

Memory Configuration			
Name	Origin	Length	Attributes
ROM	0x00000000	0x000007d0	
RAM	0x00000000	0x000007d0	
default	0x00000000	0xffffffff	
Linker script and memory map			
LOAD output.o			
.text.main	0x00000000	0x58	
*(.text)	0x00000000		. = 0x0
.text	0x00000000	0x0 output.o	
	0x00000000		end = .
	0x00000000		. = ALIGN (0x4)
.text.main	0x00000000	0x58 output.o	
	0x00000000	main	
.text			
*(.text)			
.text.test_addition	0x00000058	0x68	
.text.test_addition	0x00000058	0x68 output.o	
	0x00000058	test_addition	
.text.test_subtraction	0x000000c0	0x64	
.text.test_subtraction	0x000000c0	0x64 output.o	
	0x000000c0	test_subtraction	
.text.test_logical_operations	0x00000124	0x78	
.text.test_logical_operations	0x00000124	0x78 output.o	
	0x00000124	test_logical_operations	
.text.test_shift_operations	0x0000019c	0x58	
.text.test_shift_operations	0x0000019c	0x58 output.o	
	0x0000019c	test_shift_operations	

5- generate the dump file

```
riscv-none-elf-objdump -d output.elf > dumpfile
```

“riscv-none-elf-objdump -d output.elf > dumpfile”

This line code generate the dump file that show as the assembly code.

```
00000000 <test_addition>:
 0: fe010113      addi    sp,sp,-32
 4: 00112e23      sw      ra,28(sp)
 8: 00812c23      sw      s0,24(sp)
 c: 02010413      addi    s0,sp,32
10: 06400793      li      a5,100
14: fef42623      sw      a5,-20(s0)
18: 0c800793      li      a5,200
1c: fef42423      sw      a5,-24(s0)
20: fec42703      lw      a4,-20(s0)
24: fe842783      lw      a5,-24(s0)
28: 00f707b3      add     a5,a4,a5
2c: fef42223      sw      a5,-28(s0)
30: 800007b7      lui     a5,0x80000
34: fff78793      addi    a5,a5,-1 # 7fffffff <end+0x7ffffd77>
38: fef42623      sw      a5,-20(s0)
3c: 00100793      li      a5,1
40: fef42423      sw      a5,-24(s0)
44: fec42703      lw      a4,-20(s0)
48: fe842783      lw      a5,-24(s0)
4c: 00f707b3      add     a5,a4,a5
50: fef42223      sw      a5,-28(s0)
54: 00000013      nop
58: 01c12083      lw      ra,28(sp)
5c: 01812403      lw      s0,24(sp)
60: 02010113      addi    sp,sp,32
64: 00008067      ret

00000068 <test_subtraction>:
68: fe010113      addi    sp,sp,-32
6c: 00112e23      sw      ra,28(sp)
70: 00812c23      sw      s0,24(sp)
74: 02010413      addi    s0,sp,32
78: 0c800793      li      a5,200
7c: fef42623      sw      a5,-20(s0)
80: 06400793      li      a5,100
84: fef42423      sw      a5,-24(s0)
88: fec42703      lw      a4,-20(s0)
8c: fe842783      lw      a5,-24(s0)
90: 40f707b3      sub     a5,a4,a5
94: fef42223      sw      a5,-28(s0)
98: 800007b7      lui     a5,0x80000
9c: fef42623      sw      a5,-20(s0)
a0: 00100793      li      a5,1
a4: fef42423      sw      a5,-24(s0)
a8: fec42703      lw      a4,-20(s0)
ac: fe842783      lw      a5,-24(s0)
b0: 40f707b3      sub     a5,a4,a5
b4: fef42223      sw      a5,-28(s0)
b8: 00000013      nop
```

6- generate the binary file

```
riscv-none-elf-objcopy -O binary output.elf output.bin
```

“riscv-none-elf-objcopy -O binary output.elf output.bin”

7- generate the hex file by using a python code

```
>python makehex.py output.bin 2000 > output.hex
```

“python makehex.py output.bin 2000 > output.hex”

The output file will be the machine code of the C-code

```
1 ff010113
2 00112623
3 00812423
4 01010413
5 00000097
6 048080e7
7 00000097
8 0a8080e7
9 00000097
10 104080e7
11 00000097
12 174080e7
13 00000097
14 1c4080e7
15 00000097
16 1f0080e7
17 00000793
18 00078513
19 00c12083
20 00812403
21 01010113
22 00210063
23 fe010113
24 00112e23
25 00812c23
26 02010413
27 06400793
28 fef42623
29 0c800793
30 fef42423
31 fec42703
32 fe842783
33 00f707b3
34 fef42223
35 800007b7
36 fff78793
37 fef42623
38 00100793
39 fef42423
40 fec42703
41 fe842783
42 00f707b3
43 fef42223
44 00000013
45 01c12083
```

```
49 fe010113
50 00112e23
51 00812c23
52 02010413
53 0c800793
54 fef42623
55 06400793
56 fef42423
57 fec42703
58 fe842783
59 40f707b3
60 fef42223
61 800007b7
62 fef42623
63 00100793
64 fef42423
65 fec42703
66 fe842783
67 40f707b3
68 fef42223
69 00000013
70 01c12083
71 01812403
72 02010113
73 00008067
74 fe010113
75 00112e23
76 00812c23
77 02010413
78 f0f0f7b7
79 0f078793
80 fef42623
81 0f0f17b7
82 f0f78793
83 fef42423
84 fec42703
85 fe842783
86 00f777b3
87 fef42223
88 fec42703
89 fe842783
90 00f767b3
91 fef42223
92 fec42703
93 fe842783
94 00f747b3
95 fef42223
```

The second code:

```
00000000 <main>:
 0: ff010113      addi    sp,sp,-16
 4: 00112623      sw      ra,12(sp)
 8: 00812423      sw      s0,8(sp)
 c: 01010413      addi    s0,sp,16
10: 00000097      auipc   ra,0x0
14: 048080e7      jalr    72(ra) # 58 <test_addition>
18: 00000097      auipc   ra,0x0
1c: 0a8080e7      jalr    168(ra) # c0 <test_subtraction>
20: 00000097      auipc   ra,0x0
24: 104080e7      jalr    260(ra) # 124 <test_logical_operations>
28: 00000097      auipc   ra,0x0
2c: 174080e7      jalr    372(ra) # 19c <test_shift_operations>
30: 00000097      auipc   ra,0x0
34: 1c4080e7      jalr    452(ra) # 1f4 <test_comparisons>
38: 00000097      auipc   ra,0x0
3c: 1f0080e7      jalr    496(ra) # 228 <test_branching>
40: 00000793      li      a5,0
44: 00078513      mv      a0,a5
48: 00c12083      lw      ra,12(sp)
4c: 00812403      lw      s0,8(sp)
50: 01010113      addi    sp,sp,16
54: 00008067      ret
```

```
00000058 <test_addition>:
58: fe010113      addi    sp,sp,-32
5c: 00112e23      sw      ra,28(sp)
60: 00812c23      sw      s0,24(sp)
64: 02010413      addi    s0,sp,32
68: 06400793      li      a5,100
6c: fef42623      sw      a5,-20(s0)
70: 0c800793      li      a5,200
74: fef42423      sw      a5,-24(s0)
78: fec42703      lw      a4,-20(s0)
7c: fe842783      lw      a5,-24(s0)
80: 00f707b3      add     a5,a4,a5
84: fef42223      sw      a5,-28(s0)
88: 800007b7      lui     a5,0x80000
8c: fff78793      addi    a5,a5,-1 # 7fffffff <test_branching+0x7ffffdd7>
90: fef42623      sw      a5,-20(s0)
94: 00100793      li      a5,1
98: fef42423      sw      a5,-24(s0)
9c: fec42703      lw      a4,-20(s0)
a0: fe842783      lw      a5,-24(s0)
a4: 00f707b3      add     a5,a4,a5
a8: fef42223      sw      a5,-28(s0)
ac: 00000013      nop
b0: 01c12083      lw      ra,28(sp)
b4: 01812403      lw      s0,24(sp)
b8: 02010113      addi    sp,sp,32
bc: 00008067      ret
```

```

000000c0 <test_subtraction>:
c0:  fe010113      addi    sp,sp,-32
c4:  00112e23      sw      ra,28(sp)
c8:  00812c23      sw      s0,24(sp)
cc:  02010413      addi    s0,sp,32
d0:  0c800793      li      a5,200
d4:  fef42623      sw      a5,-20(s0)
d8:  06400793      li      a5,100
dc:  fef42423      sw      a5,-24(s0)
e0:  fec42703      lw      a4,-20(s0)
e4:  fe842783      lw      a5,-24(s0)
e8:  40f707b3      sub     a5,a4,a5
ec:  fef42223      sw      a5,-28(s0)
f0:  800007b7      lui     a5,0x80000
f4:  fef42623      sw      a5,-20(s0)
f8:  00100793      li      a5,1
fc:  fef42423      sw      a5,-24(s0)
100: fec42703      lw      a4,-20(s0)
104: fe842783      lw      a5,-24(s0)
108: 40f707b3      sub     a5,a4,a5
10c: fef42223      sw      a5,-28(s0)
110: 00000013      nop
114: 01c12083      lw      ra,28(sp)
118: 01812403      lw      s0,24(sp)
11c: 02010113      addi    sp,sp,32
120: 00008067      ret

```

```

00000124 <test_logical_operations>:
124:  fe010113      addi    sp,sp,-32
128:  00112e23      sw      ra,28(sp)
12c:  00812c23      sw      s0,24(sp)
130:  02010413      addi    s0,sp,32
134:  f0f0f7b7      lui     a5,0xf0f0f
138:  0f078793      addi    a5,a5,240 # f0f0f0f0 <test_branching+0xf0f0eec8>
13c:  fef42623      sw      a5,-20(s0)
140:  0f0f17b7      lui     a5,0xf0f1
144:  f0f78793      addi    a5,a5,-241 # f0f0f0f <test_branching+0xf0f0ce7>
148:  fef42423      sw      a5,-24(s0)
14c:  fec42703      lw      a4,-20(s0)
150:  fe842783      lw      a5,-24(s0)
154:  00f777b3      and     a5,a4,a5
158:  fef42223      sw      a5,-28(s0)
15c:  fec42703      lw      a4,-20(s0)
160:  fe842783      lw      a5,-24(s0)
164:  00f767b3      or      a5,a4,a5
168:  fef42223      sw      a5,-28(s0)
16c:  fec42703      lw      a4,-20(s0)
170:  fe842783      lw      a5,-24(s0)
174:  00f747b3      xor     a5,a4,a5
178:  fef42223      sw      a5,-28(s0)
17c:  fec42783      lw      a5,-20(s0)
180:  fff7c793      not     a5,a5
184:  fef42223      sw      a5,-28(s0)
188:  00000013      nop
18c:  01c12083      lw      ra,28(sp)
190:  01812403      lw      s0,24(sp)
194:  02010113      addi    sp,sp,32
198:  00008067      ret

```

```

0000019c <test_shift_operations>:
19c:    fe010113          addi    sp,sp,-32
1a0:    00112e23          sw      ra,28(sp)
1a4:    00812c23          sw      s0,24(sp)
1a8:    02010413          addi    s0,sp,32
1ac:    00100793          li      a5,1
1b0:    fef42623          sw      a5,-20(s0)
1b4:    fec42783          lw      a5,-20(s0)
1b8:    01f79793          slli    a5,a5,0x1f
1bc:    fef42423          sw      a5,-24(s0)
1c0:    fe842783          lw      a5,-24(s0)
1c4:    01f7d793          srli    a5,a5,0x1f
1c8:    fef42423          sw      a5,-24(s0)
1cc:    fff00793          li      a5,-1
1d0:    fef42223          sw      a5,-28(s0)
1d4:    fe442783          lw      a5,-28(s0)
1d8:    4017d793          srai    a5,a5,0x1
1dc:    fef42423          sw      a5,-24(s0)
1e0:    00000013          nop
1e4:    01c12083          lw      ra,28(sp)
1e8:    01812403          lw      s0,24(sp)
1ec:    02010113          addi    sp,sp,32
1f0:    00008067          ret

```

```

000001f4 <test_comparisons>:
1f4:    fe010113          addi    sp,sp,-32
1f8:    00112e23          sw      ra,28(sp)
1fc:    00812c23          sw      s0,24(sp)
200:    02010413          addi    s0,sp,32
204:    06400793          li      a5,100
208:    fef42623          sw      a5,-20(s0)
20c:    0c800793          li      a5,200
210:    fef42423          sw      a5,-24(s0)
214:    00000013          nop
218:    01c12083          lw      ra,28(sp)
21c:    01812403          lw      s0,24(sp)
220:    02010113          addi    sp,sp,32
224:    00008067          ret

```

```

00000228 <test_branching>:
228:    fe010113          addi    sp,sp,-32
22c:    00112e23          sw      ra,28(sp)
230:    00812c23          sw      s0,24(sp)
234:    02010413          addi    s0,sp,32
238:    fe042623          sw      zero,-20(s0)
23c:    00100793          li      a5,1
240:    fef42423          sw      a5,-24(s0)
244:    0200006f          j       264 <test_branching+0x3c>
248:    fec42703          lw      a4,-20(s0)
24c:    fe842783          lw      a5,-24(s0)
250:    00f07b3          add     a5,a4,a5
254:    fef42623          sw      a5,-20(s0)
258:    fe842783          lw      a5,-24(s0)
25c:    00178793          addi    a5,a5,1
260:    fef42423          sw      a5,-24(s0)
264:    fe842703          lw      a4,-24(s0)
268:    00a00793          li      a5,10
26c:    fce7dee3          bge     a5,a4,248 <test_branching+0x20>
270:    00000013          nop
274:    00000013          nop
278:    01c12083          lw      ra,28(sp)
27c:    01812403          lw      s0,24(sp)
280:    02010113          addi    sp,sp,32
284:    00008067          ret

```

The C-code

```

void test_addition() {
    int32_t a = 100;
    int32_t b = 200;
    int32_t result = a + b;

    a = INT32_MAX;
    b = 1;
    result = a + b;
}

void test_subtraction() {
    int32_t a = 200;
    int32_t b = 100;
    int32_t result = a - b;

    a = INT32_MIN;
    b = 1;
    result = a - b;
}

void test_logical_operations() {
    uint32_t a = 0xF0F0F0F0;
    uint32_t b = 0xF0F0F0F0;

    uint32_t result = a & b;

    result = a | b;

    result = a ^ b;

    result = ~a;
}

void test_shift_operations() {
    uint32_t a = 0x00000001;

    uint32_t result = a << 31;

    result = result >> 31;

    int32_t b = -1;
    result = b >> 1;
}

void test_comparisons() {
    int32_t a = 100;
    int32_t b = 200;
}

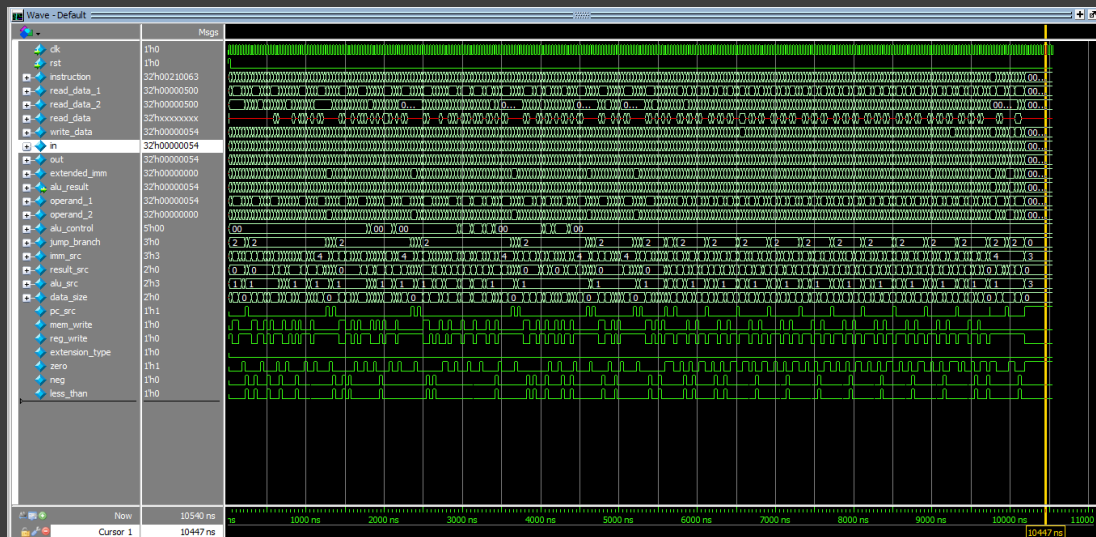
void test_branching() {
    int32_t sum = 0;
    for (int32_t i = 1; i <= 10; i++) {
        sum += i;
    }
}

int main() {
    test_addition();
    test_subtraction();
    test_logical_operations();
    test_shift_operations();
    test_comparisons();
    test_branching();

    return 0;
}

```

The simulation:



Register file contents:

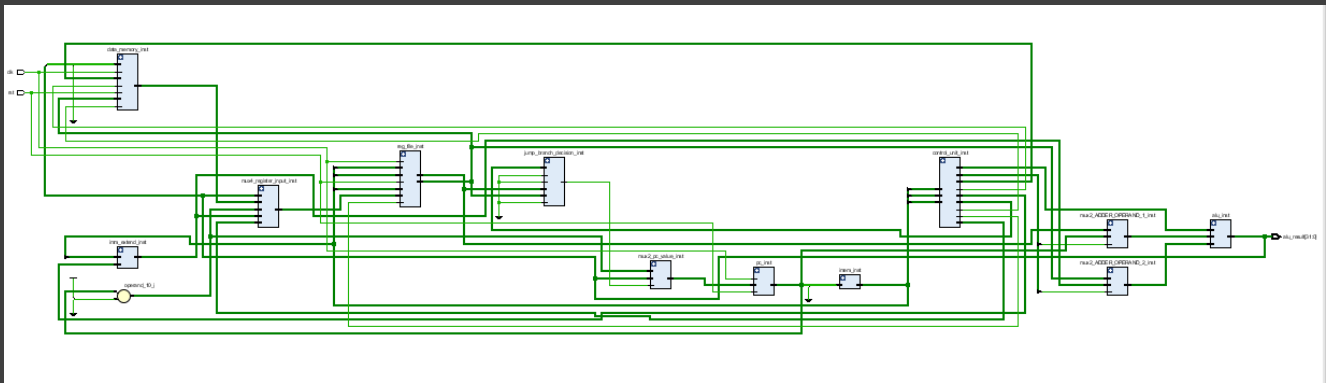
[illegible]

Data memory contents:

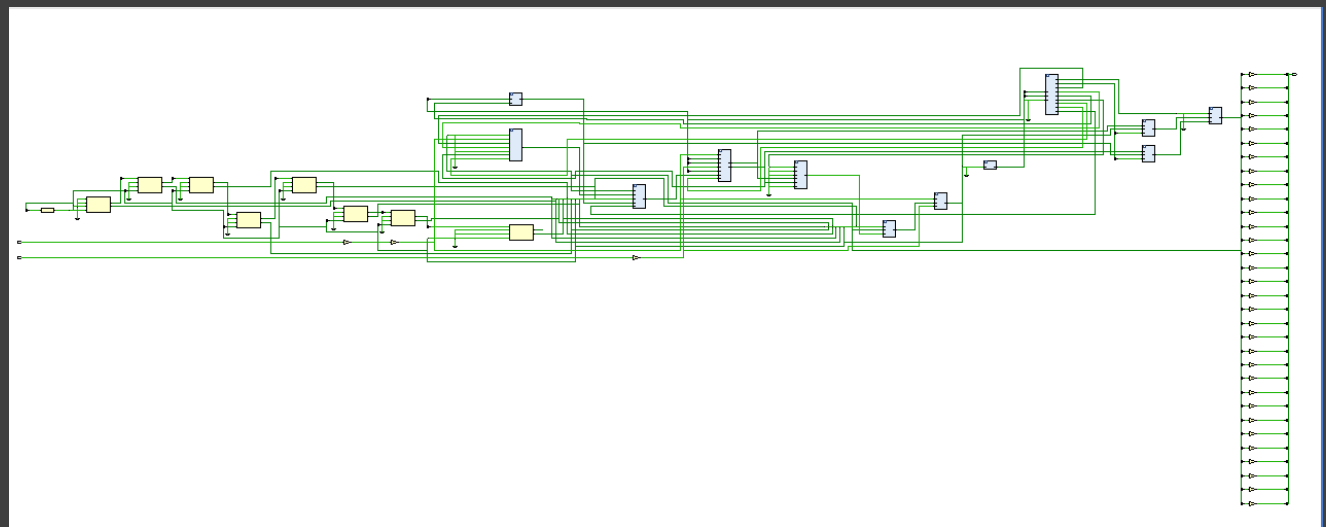
[illegible]

2.4 RTL and synthesis schematic

RTL ANALYSIS



Synthesis schematic



2.5 static-time analysis reports and utilization report

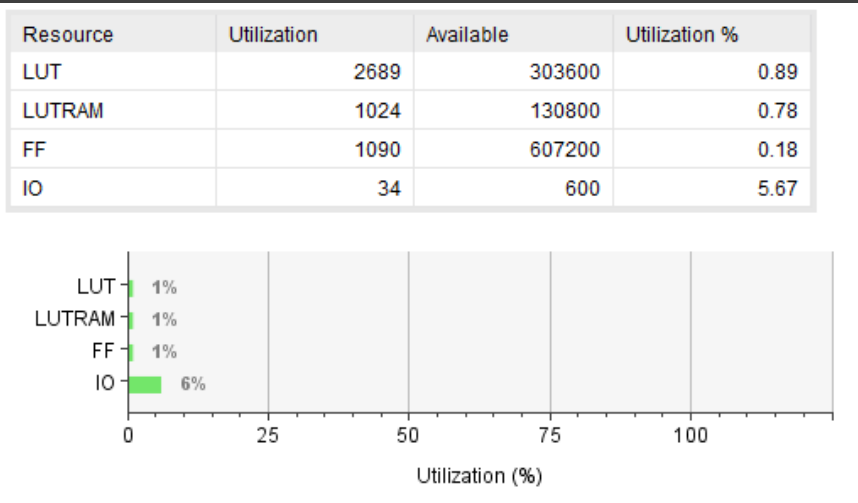
Static time analysis

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 1.419 ns	Worst Hold Slack (WHS): 0.309 ns	Worst Pulse Width Slack (WPWS): 5.870 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 12096	Total Number of Endpoints: 12096	Total Number of Endpoints: 2081

All user specified timing constraints are met.

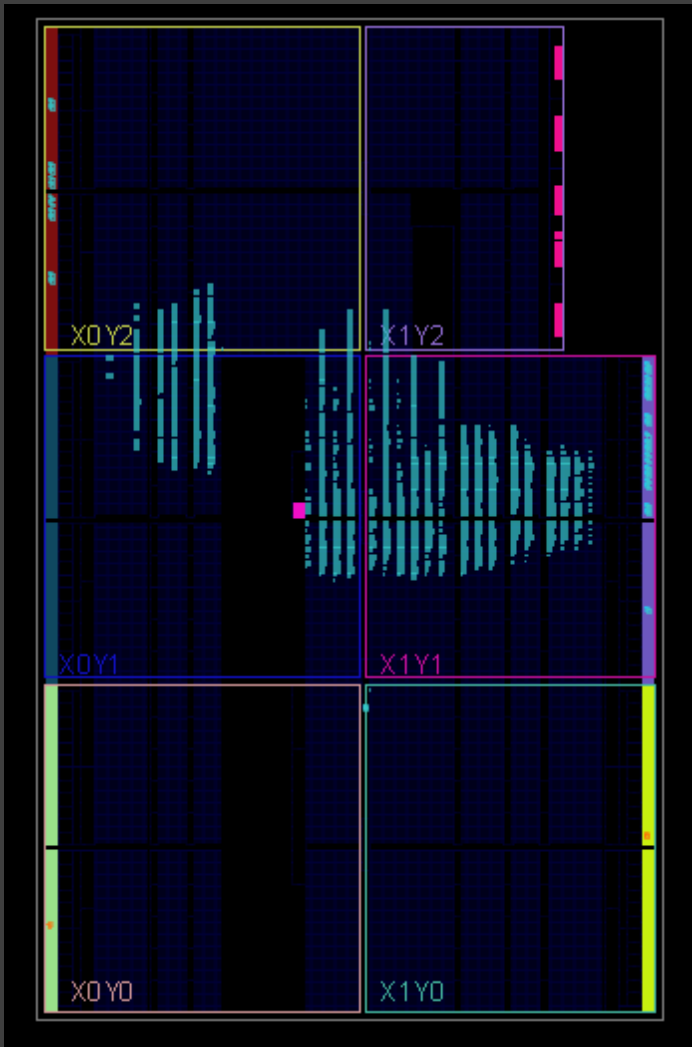
Utilization report

Name	Slice LUTs (303600)	Slice Registers (607200)	F7 Muxes (151800)	F8 Muxes (75900)	Bonded IOB (600)	BUFGCTRL (32)
▼ N top	2689	1090	736	224	34	2
❏ alu_inst (ALU)	486	0	0	0	0	0
❏ control_unit_inst (contr...	23	2	0	0	0	0
❏ data_memory_inst (da...	1314	32	480	224	0	1
❏ imem_inst (imem)	106	0	0	0	0	0
❏ imm_extend_inst (im...	25	0	0	0	0	0
❏ jump_branch_decisio...	28	0	0	0	0	0
❏ mux2_ADDER_OPER...	32	0	0	0	0	0
❏ mux2_ADDER_OPER...	16	0	0	0	0	0
❏ mux2_pc_value_inst (...)	16	0	0	0	0	0
❏ mux4_register_input_i...	32	0	0	0	0	0
❏ pc_inst (pc)	0	32	0	0	0	0
❏ reg_file_inst (reg_file)	610	1024	256	0	0	0



2.6 implementation schematic

implementation schematic



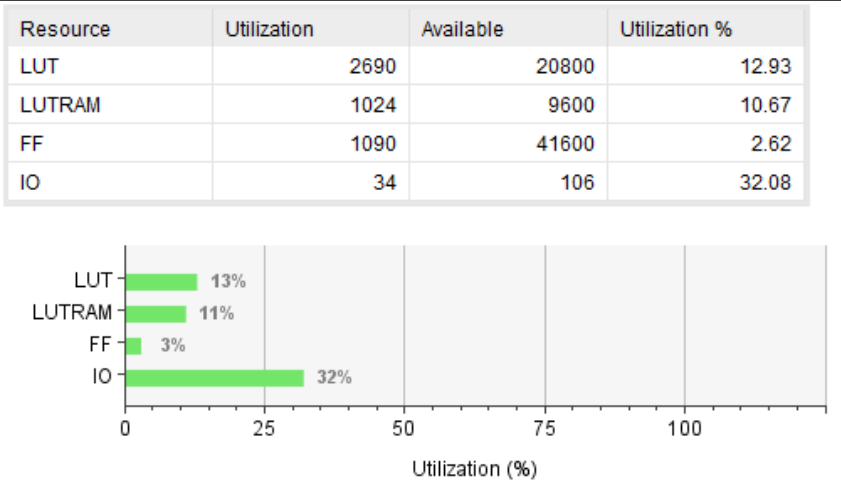
2.7 static-time analysis reports and utilization report

Static time analysis

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 0.276 ns	Worst Hold Slack (WHS): 0.388 ns	Worst Pulse Width Slack (WPWS): 6.870 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 12096	Total Number of Endpoints: 12096	Total Number of Endpoints: 2081
All user specified timing constraints are met.		

Utilization report

Name	Slice LUTs (20800)	Slice Registers (41600)	F7 Muxes (16300)	F8 Muxes (8150)	Slice (8150)	LUT as Logic (20800)	LUT as Memory (9600)	LUT Flip Flop Pairs (20800)	Bonded IOB (106)	BUFGCTRL (32)
top	2690	1090	736	224	984	1666	1024	33	34	2
alu_inst (ALU)	485	0	0	0	148	485	0	0	0	0
control_unit_inst (contr...	23	2	0	0	9	23	0	2	0	0
data_memory_inst (da...	1314	32	480	224	399	290	1024	14	0	1
imem_inst (imem)	107	0	0	0	34	107	0	0	0	0
imm_extend_inst (im...	26	0	0	0	21	26	0	0	0	0
jump_branch_decisio...	28	0	0	0	8	28	0	0	0	0
mux2_ADDER_OPER...	32	0	0	0	25	32	0	0	0	0
mux2_ADDER_OPER...	16	0	0	0	13	16	0	0	0	0
mux2_pc_value_inst (...	16	0	0	0	11	16	0	0	0	0
mux4_register_input_i...	32	0	0	0	27	32	0	0	0	0
pc_inst (pc)	0	32	0	0	11	0	0	0	0	0
reg_file_inst (reg_file)	610	1024	256	0	462	610	0	0	0	0



3. RISC-V pipeline implementation

3.1 different blocks explanation

The pipeline has the same blocks as the single-cycle implementation.

The difference is that the pipeline architecture is divided into 5 main stages that are working in parallel to each other which are:

- 1- fetch stage
- 2- decode stage
- 3- execute stage
- 4- memory stage
- 5- write-back stage

And all these stages are divided by registers that are working on the same clock.

Due to this division, there is a bunch of hazards (data and control hazards) that needs hazard control unit to solve and control the data flow.

There is also an additional block used to calculate the address of the branch and jump instructions in the decode stage instead of using the ALU in the execute stage to support branch prediction.

3.2 Verilog code

PC

```
////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
/// Name: Abdelrahman Mohamed Ragab
/// module name: pc
////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////

module pc (clk, rst, EN, in, out);

    // Base address for the first instruction
    parameter BASE_INSTRUCTION = 32'h00000000;
    // Size of the instruction (32 bits)
    parameter SIZE = 32;

    // input port
    input [SIZE-1:0] in;           // Input address (32 bits)
    input clk, rst, EN;           // Clock and reset signals

    // output port
    output reg [SIZE-1:0] out;     // Output address (32 bits)

    // Always block triggered on the rising edge of the clock
    always @(posedge clk) begin
        if (rst) begin
            // If reset is high, set the output to the base instruction address
            out <= BASE_INSTRUCTION;
        end else begin
            // If enable is high, set the output to the input value
            if (!EN) begin
                out <= in;
            end
        end
    end
end

endmodule
```

Instruction memory

```
////////////////////////////////////
// Name: Abdelrahman Mohamed Ragab
// module name: imem
////////////////////////////////////

module imem (addr, instruction);

    // Size of the address and instruction (32 bits)
    parameter SIZE = 32;
    // Base address for the instruction memory
    parameter BASE_ADDRESS = 32'h00000000;
    // Size of the instruction memory (252 MB)
    parameter mem_SIZE = 2000;                                //252 * 2**20;

    // input port
    input [SIZE-1:0] addr;                                     // Input address (32 bits)

    // output port
    output [SIZE-1:0] instruction;                             // Output instruction (32 bits)

    // Memory array (8-bit wide, mem_SIZE entries)
    reg [31:0] mem [0:mem_SIZE-1];

    // Initial block to load the instruction memory from a file
    initial begin
        // Load the memory contents from the file "imem.mem"
        $readmemh("imem.mem", mem);
    end

    // Assign the instruction output to the value stored at the specified address
    assign instruction = mem[addr - BASE_ADDRESS];

endmodule
```

ALU

```
////////////////////////////////////
// Name: Abdelrahman Mohamed Ragab
// module name: ALU
////////////////////////////////////

module ALU(operand_1, operand_2, zero, neg, less_than, result, ALU_control);

    parameter SIZE = 32;

    // input port
    input signed [SIZE-1:0] operand_1, operand_2;              // Input operands (32 bits each)
    input [4:0] ALU_control;                                    // ALU control signal (5 bits)

    // output port
    output reg signed [SIZE-1:0] result;                        // ALU result (32 bits)
    output zero, neg, less_than;                                // Zero and negative and less than flags

    // Always block to perform ALU operations based on ALU_control
    always @(*) begin
        case (ALU_control)
            5'b0000: result = operand_1 + operand_2; // ADD
            5'b0001: result = operand_1 - operand_2; // SUB
            5'b0010: result = operand_1 << operand_2; // SLL
            5'b0100: result = operand_1 < operand_2 ? 1 : 0; // SLT
            5'b0110: result = $unsigned(operand_1) < $unsigned(operand_2) ? 1 : 0; // SLTU
            5'b1000: result = operand_1 ^ operand_2; // XOR
            5'b1010: result = operand_1 >> operand_2; // SRL
            5'b1011: result = operand_1 >>> operand_2; // SRA
            5'b1100: result = operand_1 | operand_2; // OR
            5'b1110: result = operand_1 & operand_2; // AND
            default: result = 0; // Default case
        endcase
    end

    // Zero flag is high if result is zero
    assign zero = (operand_1 - operand_2 == 0);

    // Negative flag is high if result is negative
    assign neg = (operand_1 < operand_2);

    // UNSIGNED LESS THAN FLAG
    assign less_than = ($unsigned(operand_1) < $unsigned(operand_2));

endmodule
```

Extend unit

```
////////////////////////////////////
/// Name: Abdelrahman Mohamed Ragab
/// module name: imm_extend
////////////////////////////////////

module imm_extend(imm, extended_imm, imm_src);

    parameter SIZE = 32;

    // input port
    input [31:7] imm; // Input immediate value (25 bits)
    input [2:0] imm_src; // Input source type (3 bits)

    // output port
    output reg [SIZE-1:0] extended_imm; // Output extended immediate value (32 bits)

    // Select the immediate extension based on the instruction format
    always @(*) begin
        case (imm_src)
            // U-FORMAT-INSTRUCTIONS
            0: begin
                extended_imm = {imm[31:12], 12'b0};
            end
            // J-FORMAT-INSTRUCTIONS
            1: begin
                extended_imm = {{12{imm[31]}}, imm[19:12], imm[20], imm[30:21], 1'b0};
            end
            // S-FORMAT-INSTRUCTIONS
            2: begin
                extended_imm = {{20{imm[31]}}, imm[31:25], imm[11:7]};
            end
            // B-FORMAT-INSTRUCTIONS
            3: begin
                extended_imm = {{20{imm[31]}}, imm[7], imm[30:25], imm[11:8], 1'b0};
            end
            // I-FORMAT-SIGNED-INSTRUCTIONS & JALR instruction
            4: begin
                extended_imm = {{20{imm[31]}}, imm[31:20]};
            end
            // I-FORMAT-SHIFT-INSTRUCTIONS
            5: begin
                extended_imm = {{27{imm[31]}}, imm[24:20]};
            end
            // I-FORMAT-UNSIGNED-INSTRUCTIONS
            6: begin
                extended_imm = {20'b0, imm[31:20]};
            end
            // Default case for invalid instruction format
            default: extended_imm = 32'hxxxxxxxx;
        endcase
    end
endmodule
```

Branch_jump decision unit

```
////////////////////////////////////
/// Name: Abdelrahman Mohamed Ragab
/// module name: jump_branch_decision
////////////////////////////////////

module jump_branch_decision(operand_1, operand_2, zero, neg, less_than, jump_branch, pc_src);

    parameter SIZE = 32;

    // input port
    input [SIZE-1:0] operand_1, operand_2; // Input operands (32 bits each)
    input [2:0] jump_branch; // Jump or branch signal (3 bits)
    input zero, neg, less_than; // Zero and negative and less than flags

    // output port
    output reg pc_src; // Output PC source (1 bits)

    // Always block to decide the PC source

    always @(*) begin
        if (jump_branch == 3'b010) begin
            pc_src <= 0; // PC source is the incremented PC
        end else if (jump_branch == 3'b011) begin
            pc_src <= 1; // PC source is the jump target address
        end else begin
            case (jump_branch)
                3'b000: pc_src <= (zero) ? 1 : 0; // BEQ
                3'b001: pc_src <= (!zero) ? 1 : 0; // BNE
                3'b100: pc_src <= (neg) ? 1 : 0; // BLT
                3'b101: pc_src <= (!neg || zero) ? 1 : 0; // BGE
                3'b110: pc_src <= (less_than) ? 1 : 0; // BLTU
                3'b111: pc_src <= (!less_than || zero) ? 1 : 0; // BGEU
                default: pc_src <= 0; // Default case
            endcase
        end
    end
endmodule
```

Register file

```
////////////////////////////////////
// Name: Abdelrahman Mohamed Ragab
// module name: reg_file
////////////////////////////////////

module reg_file (read_address_1, read_address_2, write_address, write_data, clk, write_enable, read_data_1, read_data_2, rst);

    // Size of the register file (32 registers, each 32 bits wide)
    parameter SIZE = 32;

    // input port
    input [SIZE-1:0] write_data;
    input [SIZE-1:0] read_address_1, read_address_2, write_address;
    input clk, write_enable, rst;

    // output port
    output [SIZE-1:0] read_data_1, read_data_2;

    // Register file array (32 registers, each 32 bits wide)
    reg [SIZE-1:0] reg_file [0:SIZE-1];

    // Integer for loop iteration
    integer i;

    // Always block triggered on the rising edge of the clock
    always @(posedge clk) begin
        if (rst) begin
            // If reset is high, initialize all registers to 0
            for (i = 0; i < SIZE; i = i + 1) begin
                reg_file[i] <= 0;
            end
            reg_file[2] <= 32'h00000500;
        end else if (write_enable) begin
            // If write enable is high, write data to the specified register
            if (!write_address) begin
                reg_file[write_address] <= 0;
            end else begin
                reg_file[write_address] <= write_data;
            end
        end
    end

    // Assign the read data outputs to the values stored in the specified registers
    assign read_data_1 = reg_file[read_address_1];
    assign read_data_2 = reg_file[read_address_2];

endmodule
```

CONTROL UNIT

```
////////////////////////////////////
// Name: Abdelrahman Mohamed Ragab
// Module Name: control_unit
////////////////////////////////////

module control_unit(op, funct3, funct7, jump_branch, result_src, mem_write, alu_control, alu_src, imm_src, reg_write, data_size, extension_type);

    // input port
    input [6:0] op;
    input [6:0] funct7;
    input [2:0] funct3;

    // output port
    output reg [4:0] alu_control;
    output reg [2:0] imm_src;
    output reg [1:0] result_src;
    output reg [1:0] alu_src;
    output reg [2:0] jump_branch;
    output reg [1:0] data_size;
    output reg extension_type;
    output reg mem_write;
    output reg reg_write;

    // Always block to determine the control signals based on the opcode and funct3 and funct7
    always @(*) begin
        //alu_control
        case (op)
            // R-FORMAT-INSTRUCTIONS
            7'b0110011: case ({funct7, funct3})
                10'b00000000: alu_control <= 5'b000000; // ADD
                10'b01000000: alu_control <= 5'b000010; // SUB
                10'b00000001: alu_control <= 5'b000100; // SLL
                10'b00000010: alu_control <= 5'b010000; // SLT
                10'b00000011: alu_control <= 5'b011000; // SLTU
                10'b00000100: alu_control <= 5'b100000; // XOR
                10'b00000101: alu_control <= 5'b101000; // SRL
                10'b01000101: alu_control <= 5'b101100; // SRA
                10'b00000110: alu_control <= 5'b110000; // OR
                10'b00000111: alu_control <= 5'b111000; // AND
                default: alu_control <= 5'b000000; // Default case
            endcase
            // I-FORMAT-INSTRUCTIONS
            7'b0010011: case (funct3)
                3'b000: alu_control <= 5'b000000; // ADDI
                3'b001: alu_control <= 5'b001000; // SLLI
                3'b010: alu_control <= 5'b010000; // SLTI
                3'b011: alu_control <= 5'b011000; // SLTIU
                3'b100: alu_control <= 5'b100000; // XORI
                3'b110: alu_control <= 5'b110000; // ORI
                3'b111: alu_control <= 5'b111000; // ANDI
                3'b101: case (funct7)
                    7'b0000000: alu_control <= 5'b101000; // SRLI
                    7'b0100000: alu_control <= 5'b101010; // SRAI
                    default: alu_control <= 5'b000000; // Default case
                endcase
            endcase
            default: alu_control <= 5'b000000; // Default case
        endcase
    end
```

```

// LOAD-INSTRUCTIONS
7'b0000011: alu_control = 5'b00000; // ADD
// S-FORMAT-INSTRUCTIONS
7'b0100011: alu_control = 5'b00000; // ADD
// J-FORMAT-INSTRUCTIONS
7'b1101111, 7'b1100111: alu_control <= 5'b00000; // ADD
// U-FORMAT-INSTRUCTIONS
7'b0010111, 7'b0110111: alu_control = 5'b00000; // ADD
// B-FORMAT-INSTRUCTIONS
7'b1100011: alu_control <= 5'b00000; // ADD
// PAUSE INSTRUCTION
7'b0001111: alu_control = 5'b00000; // ADD
default: alu_control = 4'b0000;
endcase

//imm_src
case (op)
// 1-FORMAT-SIGNED-INSTRUCTIONS & JALR instruction
7'b0010011, 7'b0000011, 7'b1100111: imm_src <= 3'b100;
// S-FORMAT-INSTRUCTIONS
7'b0100011: imm_src <= 3'b010;
// B-FORMAT-INSTRUCTIONS
7'b1100011: imm_src <= 3'b011;
// U-FORMAT-INSTRUCTIONS
7'b0110111, 7'b0010111: imm_src <= 3'b000;
// JAL-INSTRUCTION
7'b1101111: imm_src <= 3'b001;
// I-FORMAT-SHIFT-INSTRUCTIONS
7'b0010011: imm_src <= 3'b101;
// I-FORMAT-UNSIGNED-INSTRUCTIONS
7'b0010011: imm_src <= 3'b110;
default: imm_src <= 3'b000;
endcase

//result_src
case (op)
// 1-FORMAT-ARITHMATIC-INSTRUCTIONS & R-FORMAT-INSTRUCTIONS & AUIPC-INSTRUCTION
7'b0010011, 7'b0110011, 7'b0010111: result_src <= 2'b00;
// I-FORMAT-LOAD-INSTRUCTIONS
7'b0000011: result_src <= 2'b01;
// JAL-INSTRUCTION & JALR-INSTRUCTION
7'b1101111, 7'b1100111: result_src <= 2'b10;
// LUI-INSTRUCTION
7'b0110111: result_src <= 2'b11;
default: result_src <= 2'b00;
endcase

//alu_src
case (op)
// R-FORMAT-INSTRUCTIONS
7'b0110011: alu_src <= 2'b00;
// I-FORMAT-INSTRUCTIONS & S-FORMAT-INSTRUCTIONS & //JALR-INSTRUCTION
7'b0000011, 7'b0010011, 7'b0100011, 7'b1100111: alu_src <= 2'b01;
// B-FORMAT-INSTRUCTIONS & JAL-INSTRUCTION & AUIPC-INSTRUCTION
7'b1101111, 7'b1100011, 7'b0010111: alu_src <= 2'b11;
default: alu_src <= 2'b00;
endcase

```

```

//jump_branch
case (op)
// JAL-INSTRUCTION & JALR-INSTRUCTION
7'b1101111, 7'b1100111: jump_branch <= 3'b011; // 3
// B-FORMAT-INSTRUCTIONS
7'b1100011: case (func3)
3'b000: jump_branch <= 3'b000; // BEQ
3'b001: jump_branch <= 3'b001; // BNE
3'b100: jump_branch <= 3'b100; // BLT
3'b101: jump_branch <= 3'b101; // BGE
3'b110: jump_branch <= 3'b110; // BLTU
3'b111: jump_branch <= 3'b111; // BGEU
default: jump_branch <= 3'b010; // NO
endcase
default: jump_branch <= 3'b010; // NO
endcase

//data_size
case (op)
7'b0000011: case (func3)
3'b000: data_size <= 2'b00; // LB
3'b001: data_size <= 2'b01; // LH
3'b100: data_size <= 2'b10; // LW
3'b101: data_size <= 2'b00; // LBU
3'b110: data_size <= 2'b01; // LHU
default: data_size <= 2'b00; // Default case
endcase
7'b0100011: case (func3)
3'b000: data_size <= 2'b00; // SB
3'b001: data_size <= 2'b01; // SH
3'b010: data_size <= 2'b10; // SW
endcase
default: data_size <= 2'b00; // Default case
endcase

//extension_type
case (op)
7'b0000011: case (func3)
3'b000, 3'b001: extension_type <= 1'b0; // LB, LH
3'b100, 3'b101: extension_type <= 1'b1; // LW, LHU
default: extension_type <= 1'b0; // Default case
endcase
default: extension_type <= 1'b0; // Default case
endcase

//mem_write
case (op)
// S-FORMAT-INSTRUCTIONS
7'b0100011: mem_write <= 1'b1;
default: mem_write <= 1'b0;
endcase

//reg_write
case (op)
// 1-FORMAT-INSTRUCTIONS & R-FORMAT-INSTRUCTIONS & JAL-INSTRUCTION & JALR-INSTRUCTION & AUIPC-INSTRUCTION
7'b0000011, 7'b0010011, 7'b0110011, 7'b1101111, 7'b1100111, 7'b0010111: reg_write <= 1'b1;
default: reg_write <= 1'b0;
endcase
end
endmodule

```

Data memory

```

////////////////////////////////////
// Name: Abdelrahman Mohamed Ragab
// Module-Name: data_memory
////////////////////////////////////

module data_memory(address, write_data, read_data, clk, rst, data_size, extension_type, write_enable);

    // Size of the address and instruction (32 bits)
    parameter SIZE = 32;
    // Base address for the instruction memory
    parameter BASE_ADDRESS = 32'h00000000;
    // Size of the instruction memory (1792 MB)
    parameter mem_SIZE = 2000; //1792 * 2**20;

    // Input port
    input [SIZE-1:0] write_data; // Input data to be written to the data memory
    input [SIZE-1:0] address; // Address for reading and writing data
    input [1:0] data_size; // Data size (2 bits)
    input clk, rst, write_enable, extension_type; // Clock, write enable, extension_type and reset signals

    // output port
    output reg [SIZE-1:0] read_data; // Data read from the specified address

    // Memory array (8-bit wide)
    reg [31:0] data_memory [0:mem_SIZE-1];

    // Integer for loop iteration
    integer i;

    // reg byte_enable;
    reg [3:0] byte_enable;

    // Always block triggered on the rising edge of the clock
    always @(*) begin
        case (address[1:0])
            0: byte_enable = 4'b0001;
            1: byte_enable = 4'b0010;
            2: byte_enable = 4'b0100;
            3: byte_enable = 4'b1000;
            default: byte_enable = 4'b0000;
        endcase
    end

    // Always block triggered on the rising edge of the clock
    always @(posedge clk) begin
        if (write_enable) begin
            // If write enable is high, write data to the specified address
            case (data_size)
                2'b00: begin
                    // Store byte
                    if (byte_enable[0]) data_memory[address - BASE_ADDRESS][7:0] <= write_data[7:0];
                    if (byte_enable[1]) data_memory[address - BASE_ADDRESS][15:8] <= write_data[7:0];
                    if (byte_enable[2]) data_memory[address - BASE_ADDRESS][23:16] <= write_data[7:0];
                    if (byte_enable[3]) data_memory[address - BASE_ADDRESS][31:24] <= write_data[7:0];
                end
                2'b01: begin
                    // Store halfword
                    if (byte_enable[0]) data_memory[address - BASE_ADDRESS][15:0] <= write_data[15:0];
                    if (byte_enable[2]) data_memory[address - BASE_ADDRESS][31:16] <= write_data[15:0];
                end
                2'b10: begin
                    // Store word
                    data_memory[address - BASE_ADDRESS] <= write_data[31:0];
                end
                default: begin
                    // Default case: store zero
                    data_memory[address - BASE_ADDRESS] <= 8'h00;
                end
            endcase
        end
    end
end

```

```

always @(*) begin
    case (data_size)
        2'b00: begin
            if (!extension_type) begin
                // Byte-Sign Extend
                if (byte_enable[0]) read_data <= {24{data_memory[address - BASE_ADDRESS][7]}}, data_memory[address - BASE_ADDRESS][7:0];
                if (byte_enable[1]) read_data <= {24{data_memory[address - BASE_ADDRESS][15]}}, data_memory[address - BASE_ADDRESS][15:8];
                if (byte_enable[2]) read_data <= {24{data_memory[address - BASE_ADDRESS][23]}}, data_memory[address - BASE_ADDRESS][23:16];
                if (byte_enable[3]) read_data <= {24{data_memory[address - BASE_ADDRESS][31]}}, data_memory[address - BASE_ADDRESS][31:24];
            end else begin
                // Byte-UNSign Extend
                if (byte_enable[0]) read_data <= {24'h000000, data_memory[address - BASE_ADDRESS][7:0]};
                if (byte_enable[1]) read_data <= {24'h000000, data_memory[address - BASE_ADDRESS][15:8]};
                if (byte_enable[2]) read_data <= {24'h000000, data_memory[address - BASE_ADDRESS][23:16]};
                if (byte_enable[3]) read_data <= {24'h000000, data_memory[address - BASE_ADDRESS][31:24]};
            end
        end
        2'b01: begin
            if (!extension_type) begin
                // Halfword-Sign Extend
                if (byte_enable[0]) read_data <= {16{data_memory[address - BASE_ADDRESS][15]}}, data_memory[address - BASE_ADDRESS][15:0];
                if (byte_enable[2]) read_data <= {16{data_memory[address - BASE_ADDRESS][31]}}, data_memory[address - BASE_ADDRESS][31:16];
            end else begin
                // Halfword-Zero Extend
                if (byte_enable[0]) read_data <= {16'h0000, data_memory[address - BASE_ADDRESS][15:0]};
                if (byte_enable[2]) read_data <= {16'h0000, data_memory[address - BASE_ADDRESS][31:16]};
            end
        end
        2'b10: read_data <= data_memory[address - BASE_ADDRESS]; // Word
        default: read_data <= 0;
    endcase
end
endmodule

```

Address calculation

```
////////////////////////////////////
// Name: Abdelrahman Mohamed Ragab
// module name: address_calculation
////////////////////////////////////

module address_calculation (pc, imm_extend, branch_jump_address);

    // input port
    input [31:0] pc, imm_extend;                                // Input PC and extended immediate value (32 bits each)

    // output port
    output reg [31:0] branch_jump_address;                      // Output branch/jump address (32 bits)

    // Always block to calculate the branch/jump address
    always @(*) begin
        // Calculate the branch/jump address by adding the PC and the extended immediate value
        branch_jump_address = pc + imm_extend;
    end
endmodule
```

fetch-decode register

```
////////////////////////////////////
// Name: Abdelrahman Mohamed Ragab
// module name: fetch_decode_register
////////////////////////////////////

module fetch_decode_register (instructionF, pcF, pc_plus_fourF, instructionD, pcD, pc_plus_fourD, clk, CLR, EN);

    // input port
    input [31:0] instructionF, pcF, pc_plus_fourF;              // Input data (32 bits)
    input clk, CLR, EN;                                         // Clock, clear, and enable signals

    // output port
    output reg [31:0] instructionD, pcD, pc_plus_fourD;         // Output data (32 bits)

    // Always block triggered on the rising edge of the clock
    always @(posedge clk) begin
        if (CLR) begin
            // If clear is high, set the output to 0
            instructionD <= 32'h00000013;
            pcD <= 32'b0;
            pc_plus_fourD <= 32'b0;
        end else if (!EN) begin
            // If enable is high, set the output to the input value
            instructionD <= instructionF;
            pcD <= pcF;
            pc_plus_fourD <= pc_plus_fourF;
        end
    end
endmodule
```

Decode-execute register

```
////////////////////////////////////
/// Name: Abdelrahman Mohamed Ragab
/// module name: decode_execute_register
////////////////////////////////////

module decode_execute_register (read_data_1_D, read_data_2_D, pcD, read_address_1_D, read_address_2_D, write_address_D, extended_imm_D, pc_plus_four_D,
jump_branch_D, result_src_D, mem_write_D, alu_control_D, alu_src_D, reg_write_D, data_size_D, extension_type_D, pc_src_prediction_D,
read_data_1_E, read_data_2_E, pcE, read_address_1_E, read_address_2_E, write_address_E, extended_imm_E, pc_plus_four_E,
jump_branch_E, result_src_E, mem_write_E, alu_control_E, alu_src_E, reg_write_E, data_size_E, extension_type_E, pc_src_prediction_E, clk, CLR, EN);

// input port
input [31:0] read_data_1_D, read_data_2_D, pcD, extended_imm_D, pc_plus_four_D;
input [4:0] read_address_1_D, read_address_2_D, write_address_D;
input [4:0] alu_control_D;
input [2:0] jump_branch_D;
input [1:0] result_src_D, alu_src_D, data_size_D;
input reg_write_D, extension_type_D, mem_write_D, pc_src_prediction_D;
input clk, CLR, EN;

// output port
output reg [31:0] read_data_1_E, read_data_2_E, pcE, extended_imm_E, pc_plus_four_E;
output reg [4:0] alu_control_E, read_address_1_E, read_address_2_E, write_address_E;
output reg [2:0] jump_branch_E;
output reg [1:0] result_src_E, alu_src_E, data_size_E;
output reg reg_write_E, extension_type_E, mem_write_E, pc_src_prediction_E;

// Always block triggered on the rising edge of the clock
always @(posedge clk) begin
    if (CLR) begin
        // If clear is high, set the output to 0
        read_data_1_E <= 32'h00000000;
        read_data_2_E <= 32'h00000000;
        pcE <= 32'h00000000;
        read_address_1_E <= 5'h00;
        read_address_2_E <= 5'h00;
        write_address_E <= 5'h00;
        extended_imm_E <= 32'h00000000;
        pc_plus_four_E <= 32'h00000000;
        jump_branch_E <= 3'h2;
        result_src_E <= 2'b00;
        mem_write_E <= 1'b0;
        alu_control_E <= 5'h00;
        alu_src_E <= 2'b00;
        reg_write_E <= 1'b0;
        data_size_E <= 2'b00;
        extension_type_E <= 1'b0;
        pc_src_prediction_E <= 1'b0;
    end
    else begin
        // If enable is high, set the output to the input value
        read_data_1_E <= read_data_1_D;
        read_data_2_E <= read_data_2_D;
        pcE <= pcD;
        read_address_1_E <= read_address_1_D;
        read_address_2_E <= read_address_2_D;
        write_address_E <= write_address_D;
        extended_imm_E <= extended_imm_D;
        pc_plus_four_E <= pc_plus_four_D;
        jump_branch_E <= jump_branch_D;
        result_src_E <= result_src_D;
        mem_write_E <= mem_write_D;
        alu_control_E <= alu_control_D;
        alu_src_E <= alu_src_D;
        reg_write_E <= reg_write_D;
        data_size_E <= data_size_D;
        extension_type_E <= extension_type_D;
        pc_src_prediction_E <= pc_src_prediction_D;
    end
end
endmodule
```

Execute-memory register

```
////////////////////////////////////
/// Name: Abdelrahman Mohamed Ragab
/// module name: execute_memory_register
////////////////////////////////////

module execute_memory_register (result_E, write_data_E, write_address_E, pc_plus_four_E, result_src_E, mem_write_E, reg_write_E, extended_imm_E, data_size_E, extension_type_E,
result_M, write_data_M, write_address_M, pc_plus_four_M, result_src_M, mem_write_M, reg_write_M, extended_imm_M, data_size_M, extension_type_M, clk);

// input port
input [31:0] result_E, write_data_E, pc_plus_four_E, extended_imm_E;
input [4:0] write_address_E;
input [1:0] result_src_E, data_size_E;
input mem_write_E, reg_write_E, extension_type_E;
input clk;

// output port
output reg [31:0] result_M, write_data_M, pc_plus_four_M, extended_imm_M;
output reg [4:0] write_address_M;
output reg [1:0] result_src_M, data_size_M;
output reg mem_write_M, reg_write_M, extension_type_M;

// Always block triggered on the rising edge of the clock
always @(posedge clk) begin
    // If enable is high, set the output to the input value
    result_M <= result_E;
    write_data_M <= write_data_E;
    write_address_M <= write_address_E;
    pc_plus_four_M <= pc_plus_four_E;
    result_src_M <= result_src_E;
    mem_write_M <= mem_write_E;
    reg_write_M <= reg_write_E;
    extended_imm_M <= extended_imm_E;
    data_size_M <= data_size_E;
    extension_type_M <= extension_type_E;
end
endmodule
```

Execute-write back register

```
//////////////////////////////////////////////////////////////////
/// Name: Abdelrahman Mohamed Ragab
/// module name: decode_execute_register
//////////////////////////////////////////////////////////////////

module decode_execute_register (read_data_1_D, read_data_2_D, pcD, read_address_1_D, read_address_2_D, write_address_D, extended_imm_D, pc_plus_four_D,
jump_branch_D, result_src_D, mem_write_D, alu_control_D, alu_src_D, reg_write_D, data_size_D, extension_type_D, pc_src_predction_D,
read_data_1_E, read_data_2_E, pcE, read_address_1_E, read_address_2_E, write_address_E, extended_imm_E, pc_plus_four_E,
jump_branch_E, result_src_E, mem_write_E, alu_control_E, alu_src_E, reg_write_E, data_size_E, extension_type_E, pc_src_predction_E, clk, CLR, EN);

// input port
input [31:0] read_data_1_D, read_data_2_D, pcD, extended_imm_D, pc_plus_four_D;
input [4:0] read_address_1_D, read_address_2_D, write_address_D;
input [4:0] alu_control_D;
input [2:0] jump_branch_D;
input [1:0] result_src_D, alu_src_D, data_size_D;
input reg_write_D, extension_type_D, mem_write_D, pc_src_predction_D;
input clk, CLR, EN;

// output port
output reg [31:0] read_data_1_E, read_data_2_E, pcE, extended_imm_E, pc_plus_four_E;
output reg [4:0] alu_control_E, read_address_1_E, read_address_2_E, write_address_E;
output reg [2:0] jump_branch_E;
output reg [1:0] result_src_E, alu_src_E, data_size_E;
output reg reg_write_E, extension_type_E, mem_write_E, pc_src_predction_E;

// Always block triggered on the rising edge of the clock
always @(posedge clk) begin
    if (CLR) begin
        // If clear is high, set the output to 0
        read_data_1_E <= 32'h00000000;
        read_data_2_E <= 32'h00000000;
        pcE <= 32'h00000000;
        read_address_1_E <= 5'h00;
        read_address_2_E <= 5'h00;
        write_address_E <= 5'h00;
        extended_imm_E <= 32'h00000000;
        pc_plus_four_E <= 32'h00000000;
        jump_branch_E <= 3'h2;
        result_src_E <= 2'b00;
        mem_write_E <= 1'b0;
        alu_control_E <= 5'h00;
        alu_src_E <= 2'b00;
        reg_write_E <= 1'b0;
        data_size_E <= 2'b00;
        extension_type_E <= 1'b0;
        pc_src_predction_E <= 1'b0;
    end
    else begin
        // If enable is high, set the output to the input value
        read_data_1_E <= read_data_1_D;
        read_data_2_E <= read_data_2_D;
        pcE <= pcD;
        read_address_1_E <= read_address_1_D;
        read_address_2_E <= read_address_2_D;
        write_address_E <= write_address_D;
        extended_imm_E <= extended_imm_D;
        pc_plus_four_E <= pc_plus_four_D;
        jump_branch_E <= jump_branch_D;
        result_src_E <= result_src_D;
        mem_write_E <= mem_write_D;
        alu_control_E <= alu_control_D;
        alu_src_E <= alu_src_D;
        reg_write_E <= reg_write_D;
        data_size_E <= data_size_D;
        extension_type_E <= extension_type_D;
        pc_src_predction_E <= pc_src_predction_D;
    end
end
endmodule
```

Execute-memory register

```
//////////////////////////////////////////////////////////////////
/// Name: Abdelrahman Mohamed Ragab
/// module name: memory_writeback_register
//////////////////////////////////////////////////////////////////

module memory_writeback_register (result_M, write_address_M, pc_plus_four_M, read_data_M, result_src_M, reg_write_M, extended_imm_M,
result_W, write_address_W, pc_plus_four_W, read_data_W, result_src_W, reg_write_W, extended_imm_W, clk);

// input port
input [31:0] result_M, pc_plus_four_M, read_data_M, extended_imm_M;
input [4:0] write_address_M;
input [1:0] result_src_M;
input reg_write_M;
input clk;

// output port
output reg [31:0] result_W, pc_plus_four_W, read_data_W, extended_imm_W;
output reg [4:0] write_address_W;
output reg [1:0] result_src_W;
output reg reg_write_W;

// Always block triggered on the rising edge of the clock
always @(posedge clk) begin
    // If enable is high, set the output to the input value
    result_W <= result_M;
    write_address_W <= write_address_M;
    pc_plus_four_W <= pc_plus_four_M;
    read_data_W <= read_data_M;
    result_src_W <= result_src_M;
    reg_write_W <= reg_write_M;
    extended_imm_W <= extended_imm_M;
end
endmodule
```

Execute-write back register

```
////////////////////////////////////////////////////////////////////////////////////////////////////////////////
/// Name: Abdelrahman Mohamed Ragab
/// module name: decode_execute_register
////////////////////////////////////////////////////////////////////////////////////////////////////////////////

module decode_execute_register (read_data_1_D, read_data_2_D, pcD, read_address_1_D, read_address_2_D, write_address_D, extended_imm_D, pc_plus_four_D,
jump_branch_D, result_src_D, mem_write_D, alu_control_D, alu_src_D, reg_write_D, data_size_D, extension_type_D, pc_src_predction_D,
read_data_1_E, read_data_2_E, pcE, read_address_1_E, read_address_2_E, write_address_E, extended_imm_E, pc_plus_four_E,
jump_branch_E, result_src_E, mem_write_E, alu_control_E, alu_src_E, reg_write_E, data_size_E, extension_type_E, pc_src_predction_E, clk, CLR, EN);

// input port
input [31:0] read_data_1_D, read_data_2_D, pcD, extended_imm_D, pc_plus_four_D;
input [4:0] read_address_1_D, read_address_2_D, write_address_D;
input [4:0] alu_control_D;
input [2:0] jump_branch_D;
input [1:0] result_src_D, alu_src_D, data_size_D;
input reg_write_D, extension_type_D, mem_write_D, pc_src_predction_D;
input clk, CLR, EN;

// output port
output reg [31:0] read_data_1_E, read_data_2_E, pcE, extended_imm_E, pc_plus_four_E;
output reg [4:0] alu_control_E, read_address_1_E, read_address_2_E, write_address_E;
output reg [2:0] jump_branch_E;
output reg [1:0] result_src_E, alu_src_E, data_size_E;
output reg reg_write_E, extension_type_E, mem_write_E, pc_src_predction_E;

// Always block triggered on the rising edge of the clock
always @(posedge clk) begin
    if (CLR) begin
        // If clear is high, set the output to 0
        read_data_1_E <= 32'h00000000;
        read_data_2_E <= 32'h00000000;
        pcE <= 32'h00000000;
        read_address_1_E <= 5'h00;
        read_address_2_E <= 5'h00;
        write_address_E <= 5'h00;
        extended_imm_E <= 32'h00000000;
        pc_plus_four_E <= 32'h00000000;
        jump_branch_E <= 3'h2;
        result_src_E <= 2'b00;
        mem_write_E <= 1'b0;
        alu_control_E <= 5'h00;
        alu_src_E <= 2'b00;
        reg_write_E <= 1'b0;
        data_size_E <= 2'b00;
        extension_type_E <= 1'b0;
        pc_src_predction_E <= 1'b0;
    end
    else begin
        // If enable is high, set the output to the input value
        read_data_1_E <= read_data_1_D;
        read_data_2_E <= read_data_2_D;
        pcE <= pcD;
        read_address_1_E <= read_address_1_D;
        read_address_2_E <= read_address_2_D;
        write_address_E <= write_address_D;
        extended_imm_E <= extended_imm_D;
        pc_plus_four_E <= pc_plus_four_D;
        jump_branch_E <= jump_branch_D;
        result_src_E <= result_src_D;
        mem_write_E <= mem_write_D;
        alu_control_E <= alu_control_D;
        alu_src_E <= alu_src_D;
        reg_write_E <= reg_write_D;
        data_size_E <= data_size_D;
        extension_type_E <= extension_type_D;
        pc_src_predction_E <= pc_src_predction_D;
    end
end
endmodule
```

Execute-memory register

```
////////////////////////////////////////////////////////////////////////////////////////////////////////////////
/// Name: Abdelrahman Mohamed Ragab
/// module name: memory_writeback_register
////////////////////////////////////////////////////////////////////////////////////////////////////////////////

module memory_writeback_register (result_M, write_address_M, pc_plus_four_M, read_data_M, result_src_M, reg_write_M, extended_imm_M,
result_W, write_address_W, pc_plus_four_W, read_data_W, result_src_W, reg_write_W, extended_imm_W, clk);

// input port
input [31:0] result_M, pc_plus_four_M, read_data_M, extended_imm_M;
input [4:0] write_address_M;
input [1:0] result_src_M;
input reg_write_M;
input clk;

// output port
output reg [31:0] result_W, pc_plus_four_W, read_data_W, extended_imm_W;
output reg [4:0] write_address_W;
output reg [1:0] result_src_W;
output reg reg_write_W;

// Always block triggered on the rising edge of the clock
always @(posedge clk) begin
    // If enable is high, set the output to the input value
    result_W <= result_M;
    write_address_W <= write_address_M;
    pc_plus_four_W <= pc_plus_four_M;
    read_data_W <= read_data_M;
    result_src_W <= result_src_M;
    reg_write_W <= reg_write_M;
    extended_imm_W <= extended_imm_M;
end
endmodule
```

Hazard control unit

```
////////////////////////////////////
// Name: Abdelrahman Mohamed Ragab
// module name: hazard_control_unit
////////////////////////////////////

module hazard_control_unit (read_address_1_D, read_address_2_D, jump_branch_D, read_address_1_E, read_address_2_E, write_address_E, pc_src_E, result_src_E, jump_branch_E, pc_src_prediction_E,
fall_through_address, branch_address_address, write_address_M, reg_write_M, write_address_W, reg_write_W,
stall_F, stall_D, flush_D, flush_E, forward_1_E, forward_2_E, pc_correct_src, correct_address, pc_src_prediction, rst, clk
);

// input port
input [4:0] read_address_1_D, read_address_2_D, read_address_1_E, read_address_2_E, write_address_E, write_address_M, write_address_W; // Addresses for reading and writing data
input [31:0] fall_through_address; // Fall through address
input [31:0] branch_address_address; // Branch address
input [2:0] jump_branch_D, jump_branch_E; // Jump or branch signal (3 bits)
input [1:0] result_src_E; // Result source
input reg_write_M, reg_write_W; // Register write enable signals
input pc_src_E, pc_src_prediction_E; // PC source
input clk, rst; // Clock and reset signals

// output port
output reg [31:0] correct_address; // Correct address
output reg [1:0] forward_1_E, forward_2_E; // Forwarding signals
output reg flush_D, flush_E; // Flush signals
output reg pc_correct_src, pc_src_prediction; // PC source signals
output reg stall_F, stall_D; // Stall signals for fetch and decode stages

// Branch prediction signals
reg [1:0] branch_prediction; // Branch prediction

// Forwarding signals
always @(*) begin
    if (((read_address_1_E == write_address_M) & reg_write_M) & (read_address_1_E != 0)) begin
        forward_1_E <= 2'b10; // Forward data from memory stage
    end else if (((read_address_1_E == write_address_W) & reg_write_W) & (read_address_1_E != 0)) begin
        forward_1_E <= 2'b01; // Forward data from writeback stage
    end else begin
        forward_1_E <= 2'b00; // No data forwarding
    end
end

always @(*) begin
    if (((read_address_2_E == write_address_M) & reg_write_M) & (read_address_2_E != 0)) begin
        forward_2_E <= 2'b10; // Forward data from memory stage
    end else if (((read_address_2_E == write_address_W) & reg_write_W) & (read_address_2_E != 0)) begin
        forward_2_E <= 2'b01; // Forward data from writeback stage
    end else begin
        forward_2_E <= 2'b00; // No data forwarding
    end
end

// Stall the fetch stage if there is a data hazard
always @(*) begin
    if (rst) begin
        stall_F <= 0;
        stall_D <= 0;
    end else if (((read_address_1_D == write_address_E) | (read_address_2_D == write_address_E)) & (result_src_E[0])) begin
        stall_F <= 1;
        stall_D <= 1;
    end else begin
        stall_F <= 0;
        stall_D <= 0;
    end
end
end
```

```
// Correct address calculation
always @(*) begin
    if (pc_src_E) begin
        correct_address <= branch_address_address;
    end else begin
        correct_address <= fall_through_address;
    end
end

// PC source prediction
always @(*) begin
    if (rst) begin
        pc_src_prediction <= 0;
    end else if (jump_branch_D != 3'b010 & branch_prediction[1]) begin
        pc_src_prediction <= 1;
    end else begin
        pc_src_prediction <= 0;
    end
end

// Branch prediction update
always @(posedge clk) begin
    if (rst) begin
        branch_prediction <= 2'b11;
    end else if (jump_branch_E != 3'b010) begin
        if (pc_src_E) begin
            case (branch_prediction)
                2'b00, 2'b01, 2'b10: branch_prediction <= branch_prediction + 1;
                default: branch_prediction <= branch_prediction;
            endcase
        end else begin
            case (branch_prediction)
                2'b11, 2'b01, 2'b10: branch_prediction <= branch_prediction - 1;
                default: branch_prediction <= branch_prediction;
            endcase
        end
    end
end
end
```

```

// Flush signals
always @(*) begin
    if (rst) begin
        flush_D <= 1;
        flush_E <= 1;
        pc_correct_src <= 0;
    end else if (jump_branch_E != 3'h2) begin
        if (pc_src_E != pc_src_predection_E) begin
            flush_D <= 1;
            flush_E <= 1;
            pc_correct_src <= 1;
        end else begin
            flush_D <= 0;
            flush_E <= 0;
            pc_correct_src <= 0;
        end
    end else begin
        flush_D <= 0;
        flush_E <= 0;
        pc_correct_src <= 0;
    end
    if (((read_address_1_D == write_address_E) | (read_address_2_D == write_address_E)) & (result_src_E[0])) begin
        flush_E <= 1;
    end
end

endmodule

```

top

```

//////////////////////////////////////////////////////////////////
// Name: Abdelrahman Mohamed Ragab
// Module-Name: top
//////////////////////////////////////////////////////////////////

module top(clk, rst, alu_result);

    // input port
    input clk; // Clock signal
    input rst; // reset signal

    // output port
    output [31:0] alu_result; // ALU result

    // wire declaration
    wire [31:0] instruction; // Instruction read from the instruction memory
    wire [31:0] read_data_1, read_data_2; // Data read from the register file
    wire [31:0] read_data_1_D, read_data_2_D; // Data read from the register file
    wire [31:0] read_data_1_E, read_data_2_E; // Data read from the register file
    wire [31:0] read_data; // Data read from the data memory
    wire [31:0] write_data; // Data to be written to the data memory
    wire [31:0] in; // input address from the program counter
    wire [31:0] out; // Output address from the program counter
    wire [31:0] extended_imm; // Extended immediate value
    wire [31:0] operand_1; // ALU operand 1
    wire [31:0] operand_2; // ALU operand 2
    wire [31:0] operand_1_reg; // Forwarded operand 1
    wire [31:0] operand_2_reg; // Forwarded operand 2
    wire [31:0] result_M; // ALU result in memory stage
    wire [31:0] write_data_M; // Data to be written to memory in memory stage
    wire [31:0] read_data_M; // Data read from memory in writeback stage
    wire [31:0] result_W; // ALU result in writeback stage
    wire [31:0] pcD, pcE; // Program counter values in decode and execute stages
    wire [31:0] pc_plus_four_D, pc_plus_four_E, pc_plus_four_M, pc_plus_four_W; // PC + 4 values in different stages
    wire [31:0] extended_imm_E, extended_imm_M, extended_imm_W; // Extended immediate values in different stages
    wire [31:0] branch_jump_address; // Branch or jump address
    wire [31:0] correct_address; // Correct address for PC
    wire [31:0] instructionD; // Instruction in decode stage
    wire [31:0] instruction_address; // Instruction address
    wire [4:0] alu_control; // ALU control signal
    wire [4:0] alu_control_D, alu_control_E; // ALU control signal
    wire [4:0] read_address_1_E, read_address_2_E, write_address_E; // Register addresses in execute stage
    wire [4:0] write_address_M, write_address_W; // Register addresses in memory and writeback stages
    wire [2:0] jump_branch; // Jump or branch signal
    wire [2:0] jump_branch_D, jump_branch_E; // Jump or branch signal
    wire [2:0] imm_src; // Immediate source
    wire [1:0] result_src, alu_src, alu_src_E, alu_src_D; // Result source and ALU source
    wire [1:0] result_src_E, result_src_M, result_src_W; // Result source in different stages
    wire [1:0] data_size, data_size_E, data_size_M; // Data size in different stages
    wire [1:0] forward_1_E, forward_2_E; // Forwarding control signals
    wire pc_src, mem_write, reg_write; // Control signals for program counter, data memory, and register file
    wire mem_write_E, mem_write_M; // Memory write control signals in different stages
    wire reg_write_E, reg_write_M, reg_write_W; // Register write control signals in different stages
    wire extension_type; // Extension type
    wire extension_type_E, extension_type_M; // Extension type in different stages
    wire zero, neg, less_than; // Zero, negative, and less than flags
    wire flush_D, flush_E; // Clear and flush signals
    wire stall_F, stall_D; // Stall signals
    wire pc_correct_src, pc_src_predection, pc_src_predection_E; // PC source control signals

```

```
// module instantiation

// fetch_decode_register
fetch_decode_register fetch_decode_register_inst (.instructionF(instruction), .pcF(instruction_address), .pc_plus_fourF(out + 4), .instructionD(instructionD), .pcD(pcD),
.pc_plus_fourD(pc_plus_four_D), .clk(clk), .CLR(flush_D), .EN(stall_D));

// decode_execute_register
decode_execute_register decode_execute_register_inst (.read_data_1D(read_data_1), .read_data_2D(read_data_2), .pcD(pcD), .read_address_1D(instructionD[19:15]),
.read_address_2D(instructionD[24:28]), .write_address_D(instructionD[31:7]), .extended_imm_D(extended_imm), .pc_plus_four_D(pc_plus_four_D), .jump_branch_D(jump_branch),
.result_src_D(result_src), .mem_write_D(mem_write), .alu_control_D(alu_control), .alu_src_D(alu_src), .reg_write_D(reg_write), .data_size_D(data_size),
.extension_type_D(extension_type), .pc_src_prediction_D(pc_src_prediction), .read_data_1_E(read_data_1_E), .read_data_2_E(read_data_2_E), .pcF(pcF), .read_address_1_E(read_address_1_E), .read_address_2_E(read_address_2_E),
.write_address_E(write_address_E), .extended_imm_E(extended_imm_E), .pc_plus_four_E(pc_plus_four_E), .jump_branch_E(jump_branch_E), .result_src_E(result_src_E), .mem_write_E(mem_write_E),
.alu_control_E(alu_control_E), .alu_src_E(alu_src_E), .reg_write_E(reg_write_E), .data_size_E(data_size_E), .extension_type_E(extension_type_E), .pc_src_prediction_E(pc_src_prediction_E), .clk(clk), .CLR(flush_E), .EN(stall_D));

// mux to select the next forwardA register value
mux1 mux1_forwardA_register_inst(.operand_1(read_data_1_E), .operand_2(write_data), .operand_3(result_M), .result(operand_1_reg), .select(forward_1_E));

// mux to select the next forwardB register value
mux3 mux3_forwardB_register_inst(operand_1(read_data_2_E), .operand_2(write_data), .operand_3(result_M), .result(operand_2_reg), .select(forward_2_E));

// execute_memory_register
execute_memory_register execute_memory_register_inst (.result_E(alu_result), .write_data_E(operand_2_reg), .write_address_E(write_address_E), .pc_plus_four_E(pc_plus_four_E),
.result_src_E(result_src_E), .mem_write_E(mem_write_E), .reg_write_E(reg_write_E), .extended_imm_E(extended_imm_E), .data_size_E(data_size_E), .extension_type_E(extension_type_E),
.result_M(result_M), .write_data_M(write_data_M), .write_address_M(write_address_M), .pc_plus_four_M(pc_plus_four_M), .result_src_M(result_src_M),
.mem_write_M(mem_write_M), .reg_write_M(reg_write_M), .extended_imm_M(extended_imm_M), .data_size_M(data_size_M), .extension_type_M(extension_type_M), .clk(clk));

//memory_writeback_register
memory_writeback_register memory_writeback_register_inst (.result_M(result_M), .write_address_M(write_address_M), .pc_plus_four_M(pc_plus_four_M), .read_data_M(read_data),
.result_src_M(result_src_M), .reg_write_M(reg_write_M), .extended_imm_M(extended_imm_M), .result_W(result_W), .write_address_W(write_address_W), .pc_plus_four_W(pc_plus_four_W), .read_data_W(read_data_W),
.result_src_W(result_src_W), .reg_write_W(reg_write_W), .extended_imm_W(extended_imm_W), .clk(clk));

//address_calculation
address_calculation address_calculation_inst (.pc(pcD), .imm_extend(extended_imm), .branch_jump_address(branch_jump_address));

// Hazard Control Unit
hazard_control_unit hazard_control_unit_inst(.read_address_1_D(instructionD[19:15]), .read_address_2_D(instructionD[24:28]), .jump_branch_D(jump_branch), .read_address_1_E(read_address_1_E),
.read_address_2_E(read_address_2_E), .write_address_E(write_address_E), .pc_src_E(pc_src), .result_src_E(result_src_E), .jump_branch_E(jump_branch_E), .pc_src_prediction_E(pc_src_prediction_E),
.fall_through_address(pcE + 4), .branch_address_address(branch_jump_address), .write_address_M(write_address_M), .reg_write_M(reg_write_M), .write_address_W(write_address_W), .reg_write_W(reg_write_W),
.stall_F(stall_F), .stall_D(stall_D), .flush_D(flush_D), .flush_E(flush_E), .forward_1_E(forward_1_E), .forward_2_E(forward_2_E), .pc_correct_src(pc_correct_src), .correct_address(correct_address),
.pc_src_prediction(pc_src_prediction), .rst(rst), .clk(clk));

// Mux to select the next PC value
mux2 mux2_pc_value_inst(.operand_1(instruction_address + 4), .operand_2(correct_address), .result(in), .select(pc_correct_src));

// Program Counter
pc pc_inst(.clk(clk), .rst(rst), .in(in), .out(out), .EN(stall_F));

//Mux to select the next branch address
mux2 mux2_branch_address_inst(.operand_1(out), .operand_2(branch_jump_address), .result(instruction_address), .select(pc_src_prediction));

// Instruction Memory
mem mem_inst(.addr((2'b00,instruction_address[31:2])), .instruction(instruction));

// Mux to select the register input value
mux4 mux4_register_input_inst(.operand_1(result_W), .operand_2(read_data_W), .operand_3(pc_plus_four_W), .operand_4(extended_imm_W), .result(write_data), .select(result_src_W));

// Register File
reg_file reg_file_inst(.read_address_1(instructionD[19:15]), .read_address_2(instructionD[24:28]), .write_address(write_address_W), .write_data(write_data), .clk(clk),
.write_enable(reg_write_W), .read_data_1(read_data_1), .read_data_2(read_data_2), .rst(rst));

// Mux to select the first operand for the ALU
mux2 mux2_ADDER_OPERAND_1_inst(.operand_1(operand_1_reg), .operand_2(pcE), .result(operand_1), .select(alu_src_E[1]));

// Mux to select the second operand for the ALU
mux2 mux2_ADDER_OPERAND_2_inst(.operand_1(operand_2_reg), .operand_2(extended_imm_E), .result(operand_2), .select(alu_src_E[0]));

// ALU
ALU alu_inst(.operand_1(operand_1), .operand_2(operand_2), .zero(zero), .neg(neg), .less_than(less_than), .result(alu_result), .ALU_control(alu_control_E));

// Data Memory
data_memory data_memory_inst(.address((2'b00,result_M[31:2])), .write_data(write_data_M), .read_data(read_data), .clk(clk), .rst(rst),
.data_size(data_size_M), .extension_type(extension_type_M), .write_enable(mem_write_M));

// Immediate Value Extender
imm_extend imm_extend_inst(.imm(instructionD[31:7]), .extended_imm(extended_imm), .imm_src(imm_src));

// Control Unit
control_unit control_unit_inst(.op(instructionD[6:0]), .func3(instructionD[14:12]), .func7(instructionD[31:25]), .jump_branch(jump_branch), .result_src(result_src), .mem_write(mem_write), .alu_control(alu_control), .alu_src(alu_src),
.imm_src(imm_src), .reg_write(reg_write), .data_size(data_size), .extension_type(extension_type));

// Jump and Branch Decision Unit
jump_branch_decision jump_branch_decision_inst(.operand_1(read_data_1_E), .operand_2(read_data_2_E), .zero(zero), .neg(neg), .less_than(less_than), .jump_branch(jump_branch_E), .pc_src(pc_src));

endmodule
```

The above code represents the pipelined RISCv processor with all the base integer instructions signed and unsigned with the same functions mentioned above for each block.

3.3 simulation and code compilation

For verification the code functionality I used the same codes I used for the single cycle RISCV.

The first code:

#	RISC-V Assembly	Description	Address	Machine Code
main:	addi x2, x0, 5	# x2 = 5	0	00500113
	addi x3, x0, 12	# x3 = 12	4	00C00193
	addi x7, x3, -9	# x7 = (12 - 9) = 3	8	FF718393
	or x4, x7, x2	# x4 = (3 OR 5) = 7	C	0023E233
	and x5, x3, x4	# x5 = (12 AND 7) = 4	10	0041F2B3
	add x5, x5, x4	# x5 = 4 + 7 = 11	14	004282B3
	beq x5, x7, end	# shouldn't be taken	18	02728863
	slt x4, x3, x4	# x4 = (12 < 7) = 0	1C	0041A233
	beq x4, x0, around	# should be taken	20	00020463
	addi x5, x0, 0	# shouldn't execute	24	00000293
around:	slt x4, x7, x2	# x4 = (3 < 5) = 1	28	0023A233
	add x7, x4, x5	# x7 = (1 + 11) = 12	2C	005203B3
	sub x7, x7, x2	# x7 = (12 - 5) = 7	30	402383B3
	sw x7, 84(x3)	# [96] = 7	34	0471AA23
	lw x2, 96(x0)	# x2 = [96] = 7	38	06002103
	add x9, x2, x5	# x9 = (7 + 11) = 18	3C	005104B3
	jal x3, end	# jump to end, x3 = 0x44	40	008001EF
	addi x2, x0, 1	# shouldn't execute	44	00100113
end:	add x2, x2, x9	# x2 = (7 + 18) = 25	48	00910133
	sw x2, 0x20(x3)	# [100] = 25	4C	0221A023
done:	beq x2, x2, done	# infinite loop	50	00210063

Figure 7.64 riscvtest.s

The simulation:



Register file contents:

[illegible]

Data memory contents:

```
00000000 XXXXXXXX XXXXXXXX XXXXXXXX XXXXXXXX XXXXXXXX XXXXXXXX
00000014 XXXXXXXX XXXXXXXX XXXXXXXX XXXXXXXX 00000007 00000019
00000028 XXXXXXXX XXXXXXXX XXXXXXXX XXXXXXXX XXXXXXXX XXXXXXXX
0000003c XXXXXXXX XXXXXXXX XXXXXXXX XXXXXXXX XXXXXXXX XXXXXXXX
```

As we can see the values inside the data memory and the register file is the same as the values written in the reference and is the same as the single cycle values.

The second code:

```
00000000 <main>:
 0: ff010113      addi    sp,sp,-16
 4: 00112623      sw      ra,12(sp)
 8: 00812423      sw      s0,8(sp)
 c: 01010413      addi    s0,sp,16
10: 00000097      auipc   ra,0x0
14: 048080e7      jalr    72(ra) # 58 <test_addition>
18: 00000097      auipc   ra,0x0
1c: 0a8080e7      jalr    168(ra) # c0 <test_subtraction>
20: 00000097      auipc   ra,0x0
24: 104080e7      jalr    260(ra) # 124 <test_logical_operations>
28: 00000097      auipc   ra,0x0
2c: 174080e7      jalr    372(ra) # 19c <test_shift_operations>
30: 00000097      auipc   ra,0x0
34: 1c4080e7      jalr    452(ra) # 1f4 <test_comparisons>
38: 00000097      auipc   ra,0x0
3c: 1f0080e7      jalr    496(ra) # 228 <test_branching>
40: 00000793      li      a5,0
44: 00078513      mv      a0,a5
48: 00c12083      lw      ra,12(sp)
4c: 00812403      lw      s0,8(sp)
50: 01010113      addi    sp,sp,16
54: 00008067      ret
```

```
00000058 <test_addition>:
58: fe010113      addi    sp,sp,-32
5c: 00112e23      sw      ra,28(sp)
60: 00812c23      sw      s0,24(sp)
64: 02010413      addi    s0,sp,32
68: 06400793      li      a5,100
6c: fef42623      sw      a5,-20(s0)
70: 0c800793      li      a5,200
74: fef42423      sw      a5,-24(s0)
78: fec42703      lw      a4,-20(s0)
7c: fe842783      lw      a5,-24(s0)
80: 00f707b3      add     a5,a4,a5
84: fef42223      sw      a5,-28(s0)
88: 800007b7      lui     a5,0x80000
8c: fff78793      addi    a5,a5,-1 # 7fffffff <test_branching+0x7ffffdd7>
90: fef42623      sw      a5,-20(s0)
94: 00100793      li      a5,1
98: fef42423      sw      a5,-24(s0)
9c: fec42703      lw      a4,-20(s0)
a0: fe842783      lw      a5,-24(s0)
a4: 00f707b3      add     a5,a4,a5
a8: fef42223      sw      a5,-28(s0)
ac: 00000013      nop
b0: 01c12083      lw      ra,28(sp)
b4: 01812403      lw      s0,24(sp)
b8: 02010113      addi    sp,sp,32
bc: 00008067      ret
```

```

000000c0 <test_subtraction>:
c0:  fe010113          addi    sp,sp,-32
c4:  00112e23          sw      ra,28(sp)
c8:  00812c23          sw      s0,24(sp)
cc:  02010413          addi    s0,sp,32
d0:  0c800793          li      a5,200
d4:  fef42623          sw      a5,-20(s0)
d8:  06400793          li      a5,100
dc:  fef42423          sw      a5,-24(s0)
e0:  fec42703          lw      a4,-20(s0)
e4:  fe842783          lw      a5,-24(s0)
e8:  40f707b3          sub     a5,a4,a5
ec:  fef42223          sw      a5,-28(s0)
f0:  800007b7          lui     a5,0x80000
f4:  fef42623          sw      a5,-20(s0)
f8:  00100793          li      a5,1
fc:  fef42423          sw      a5,-24(s0)
100: fec42703          lw      a4,-20(s0)
104: fe842783          lw      a5,-24(s0)
108: 40f707b3          sub     a5,a4,a5
10c: fef42223          sw      a5,-28(s0)
110: 00000013          nop
114: 01c12083          lw      ra,28(sp)
118: 01812403          lw      s0,24(sp)
11c: 02010113          addi    sp,sp,32
120: 00008067          ret

```

```

00000124 <test_logical_operations>:
124:  fe010113          addi    sp,sp,-32
128:  00112e23          sw      ra,28(sp)
12c:  00812c23          sw      s0,24(sp)
130:  02010413          addi    s0,sp,32
134:  f0f0f7b7          lui     a5,0xf0f0f
138:  0f078793          addi    a5,a5,240 # f0f0f0f0 <test_branching+0xf0f0eec8>
13c:  fef42623          sw      a5,-20(s0)
140:  0f0f17b7          lui     a5,0xf0f1
144:  f0f78793          addi    a5,a5,-241 # f0f0f0f <test_branching+0xf0f0ce7>
148:  fef42423          sw      a5,-24(s0)
14c:  fec42703          lw      a4,-20(s0)
150:  fe842783          lw      a5,-24(s0)
154:  00f777b3          and     a5,a4,a5
158:  fef42223          sw      a5,-28(s0)
15c:  fec42703          lw      a4,-20(s0)
160:  fe842783          lw      a5,-24(s0)
164:  00f767b3          or      a5,a4,a5
168:  fef42223          sw      a5,-28(s0)
16c:  fec42703          lw      a4,-20(s0)
170:  fe842783          lw      a5,-24(s0)
174:  00f747b3          xor     a5,a4,a5
178:  fef42223          sw      a5,-28(s0)
17c:  fec42783          lw      a5,-20(s0)
180:  fff7c793          not     a5,a5
184:  fef42223          sw      a5,-28(s0)
188:  00000013          nop
18c:  01c12083          lw      ra,28(sp)
190:  01812403          lw      s0,24(sp)
194:  02010113          addi    sp,sp,32
198:  00008067          ret

```

```

0000019c <test_shift_operations>:
19c:  fe010113      addi    sp,sp,-32
1a0:  00112e23      sw      ra,28(sp)
1a4:  00812c23      sw      s0,24(sp)
1a8:  02010413      addi    s0,sp,32
1ac:  00100793      li      a5,1
1b0:  fef42623      sw      a5,-20(s0)
1b4:  fec42783      lw      a5,-20(s0)
1b8:  01f79793      slli    a5,a5,0x1f
1bc:  fef42423      sw      a5,-24(s0)
1c0:  fe842783      lw      a5,-24(s0)
1c4:  01f7d793      srli    a5,a5,0x1f
1c8:  fef42423      sw      a5,-24(s0)
1cc:  fff00793      li      a5,-1
1d0:  fef42223      sw      a5,-28(s0)
1d4:  fe442783      lw      a5,-28(s0)
1d8:  4017d793      srai    a5,a5,0x1
1dc:  fef42423      sw      a5,-24(s0)
1e0:  00000013      nop
1e4:  01c12083      lw      ra,28(sp)
1e8:  01812403      lw      s0,24(sp)
1ec:  02010113      addi    sp,sp,32
1f0:  00008067      ret

```

```

000001f4 <test_comparisons>:
1f4:  fe010113      addi    sp,sp,-32
1f8:  00112e23      sw      ra,28(sp)
1fc:  00812c23      sw      s0,24(sp)
200:  02010413      addi    s0,sp,32
204:  06400793      li      a5,100
208:  fef42623      sw      a5,-20(s0)
20c:  0c800793      li      a5,200
210:  fef42423      sw      a5,-24(s0)
214:  00000013      nop
218:  01c12083      lw      ra,28(sp)
21c:  01812403      lw      s0,24(sp)
220:  02010113      addi    sp,sp,32
224:  00008067      ret

```

```

00000228 <test_branching>:
228:    fe010113          addi    sp,sp,-32
22c:    00112e23          sw      ra,28(sp)
230:    00812c23          sw      s0,24(sp)
234:    02010413          addi    s0,sp,32
238:    fe042623          sw      zero,-20(s0)
23c:    00100793          li      a5,1
240:    fef42423          sw      a5,-24(s0)
244:    0200006f          j       264 <test_branching+0x3c>
248:    fec42703          lw      a4,-20(s0)
24c:    fe842783          lw      a5,-24(s0)
250:    00f07b3          add     a5,a4,a5
254:    fef42623          sw      a5,-20(s0)
258:    fe842783          lw      a5,-24(s0)
25c:    00178793          addi    a5,a5,1
260:    fef42423          sw      a5,-24(s0)
264:    fe842703          lw      a4,-24(s0)
268:    00a00793          li      a5,10
26c:    fce7dee3          bge     a5,a4,248 <test_branching+0x20>
270:    00000013          nop
274:    00000013          nop
278:    01c12083          lw      ra,28(sp)
27c:    01812403          lw      s0,24(sp)
280:    02010113          addi    sp,sp,32
284:    00008067          ret

```

The C-code

```

void test_addition() {
    int32_t a = 100;
    int32_t b = 200;
    int32_t result = a + b;

    a = INT32_MAX;
    b = 1;
    result = a + b;
}

void test_subtraction() {
    int32_t a = 200;
    int32_t b = 100;
    int32_t result = a - b;

    a = INT32_MIN;
    b = 1;
    result = a - b;
}

void test_logical_operations() {
    uint32_t a = 0xF0F0F0F0;
    uint32_t b = 0xF0F0F0F0;

    uint32_t result = a & b;

    result = a | b;

    result = a ^ b;

    result = ~a;
}

void test_shift_operations() {
    uint32_t a = 0x00000001;

    uint32_t result = a << 31;

    result = result >> 31;

    int32_t b = -1;
    result = b >> 1;
}

void test_comparisons() {
    int32_t a = 100;
    int32_t b = 200;
}

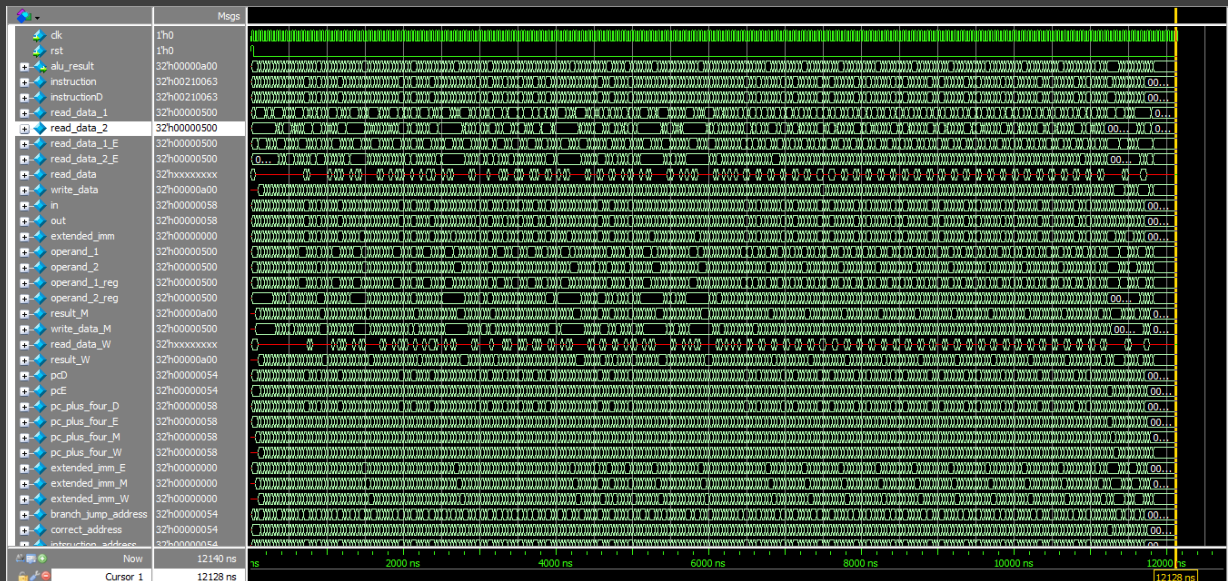
void test_branching() {
    int32_t sum = 0;
    for (int32_t i = 1; i <= 10; i++) {
        sum += i;
    }
}

int main() {
    test_addition();
    test_subtraction();
    test_logical_operations();
    test_shift_operations();
    test_comparisons();
    test_branching();

    return 0;
}

```

The simulation:



Register file contents:

[illegible]

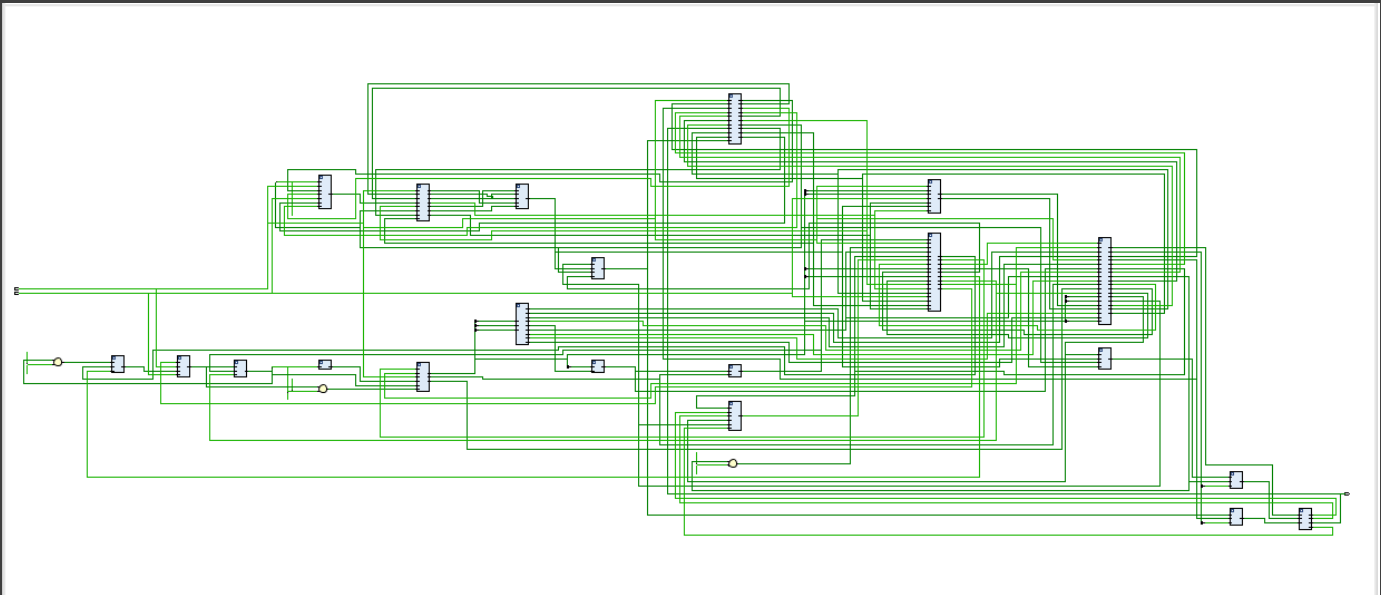
Data memory contents:

[illegible]

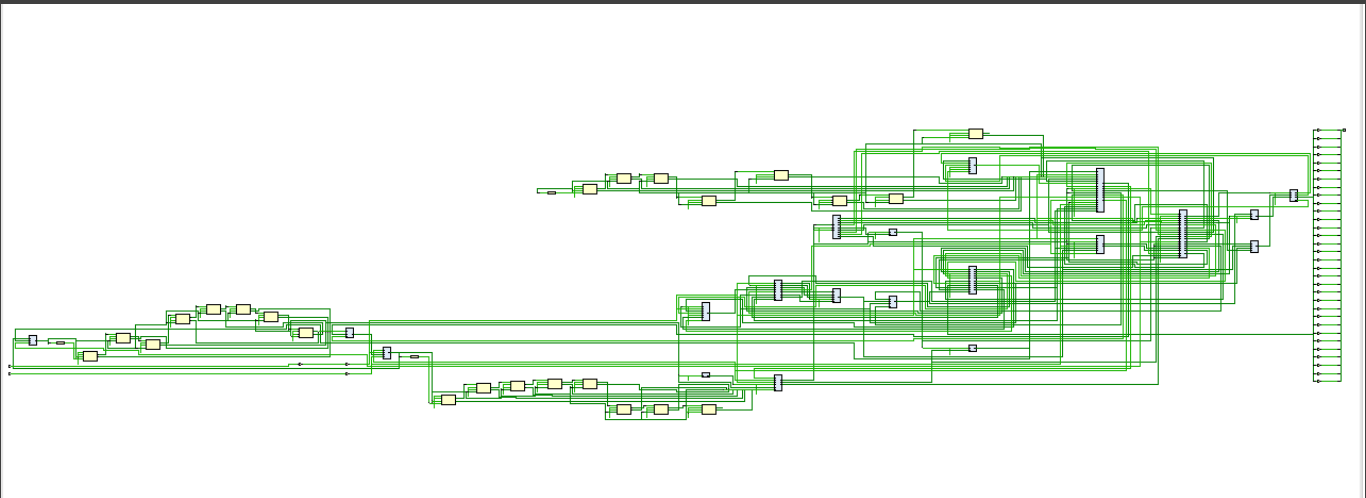
As we can see the values inside the data memory and the register file is the same as the single cycle values.

3.4 RTL and synthesis schematic

RTL ANALYSIS



Synthesis schematic



3.5 static-time analysis reports and utilization report

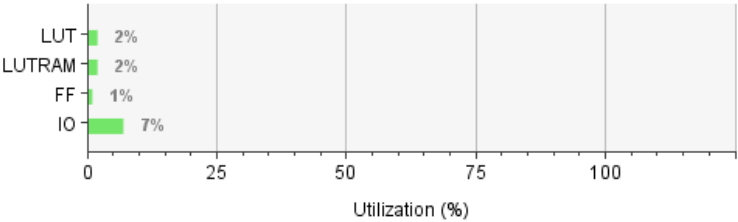
Static time analysis

Design Timing Summary			
Setup		Hold	Pulse Width
Worst Negative Slack (WNS): 3.961 ns		Worst Hold Slack (WHS): 0.119 ns	Worst Pulse Width Slack (WPWS): 5.870 ns
Total Negative Slack (TNS): 0.000 ns		Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0		Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 12983		Total Number of Endpoints: 12983	Total Number of Endpoints: 2614
All user specified timing constraints are met.			

Utilization report

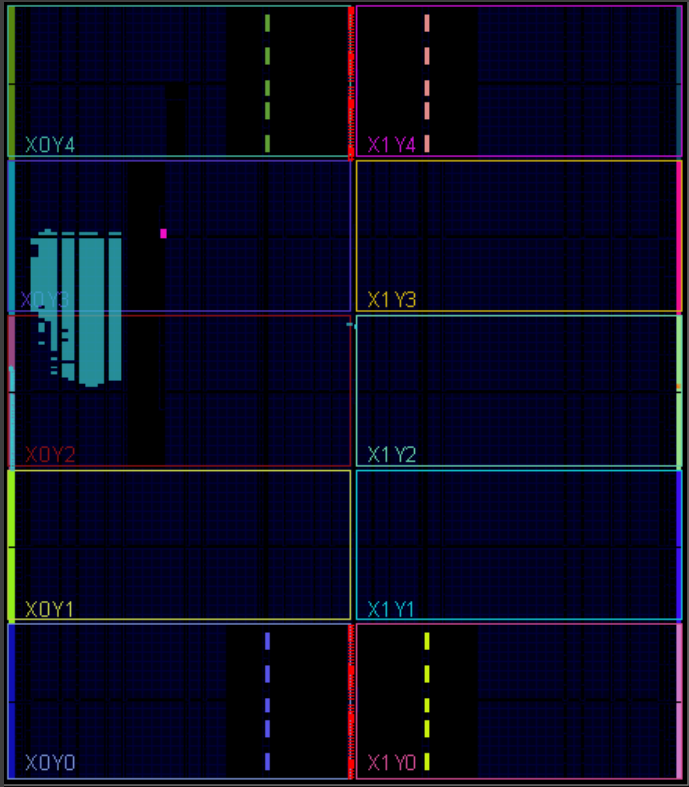
Name	Slice LUTs (134600)	Slice Registers (269200)	F7 Muxes (67300)	F8 Muxes (33650)	Bonded IOB (285)	BUFGCTRL (32)
top	2471	1623	643	224	34	2
address_calculation_i...	32	0	0	0	0	0
alu_inst (ALU)	480	0	3	0	0	0
control_unit_inst (contr...	20	2	0	0	0	0
data_memory_inst (da...	1314	32	480	224	0	1
decode_execute_regis...	0	182	0	0	0	0
execute_memory_regi...	0	133	0	0	0	0
fetch_decode_register...	1	87	0	0	0	0
hazard_control_unit_in...	42	2	0	0	0	0
imem_inst (imem)	12	0	0	0	0	0
imm_extend_inst (im...	19	0	0	0	0	0
jump_branch_decisio...	1	0	0	0	0	0
memory_writeback_re...	0	129	0	0	0	0
mux2_ADDER_OPER...	16	0	0	0	0	0
mux2_branch_addres...	32	0	0	0	0	0
mux2_pc_value_inst (...	16	0	0	0	0	0
mux3_forwardA_regist...	32	0	0	0	0	0
mux3_forwardB_regist...	32	0	0	0	0	0
mux4_register_input_i...	32	0	0	0	0	0
pc_inst (pc)	1	32	0	0	0	0
reg_file_inst (reg_file)	386	1024	160	0	0	0

Resource	Utilization	Available	Utilization %
LUT	2471	134600	1.84
LUTRAM	1024	46200	2.22
FF	1623	269200	0.60
IO	34	500	6.80



3.6 implementation schematic

implementation schematic



3.7 static-time analysis reports and utilization report

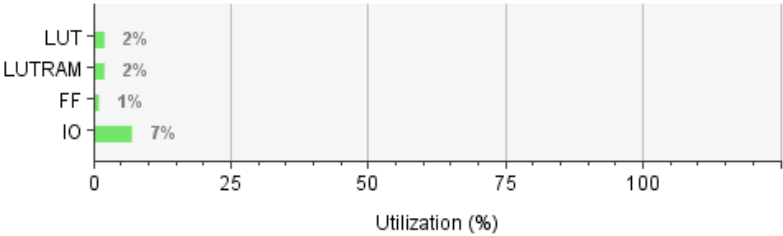
Static time analysis

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 2.248 ns	Worst Hold Slack (WHS): 0.095 ns	Worst Pulse Width Slack (WPWS): 5.870 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 12983	Total Number of Endpoints: 12983	Total Number of Endpoints: 2614
All user specified timing constraints are met.		

Utilization report

Name	Slice LUTs (133800)	Slice Registers (267600)	F7 Muxes (66900)	F8 Muxes (33450)	Slice (33450)	LUT as Logic (133800)	LUT as Memory (46200)	LUT Flip Flop Pairs (133800)	Bonded IOB (500)	BUFGCTRL (32)
top	2467	1623	643	224	892	1443	1024	173	34	2
address_calculation_inst	32	0	0	0	8	32	0	0	0	0
alu_inst (ALU)	478	0	3	0	148	478	0	0	0	0
control_unit_inst (contr...	20	2	0	0	7	20	0	2	0	0
data_memory_inst (da...	1314	32	480	224	399	290	1024	13	0	1
decode_execute_regis...	0	182	0	0	80	0	0	0	0	0
execute_memory_regi...	0	133	0	0	79	0	0	0	0	0
fetch_decode_register...	0	87	0	0	16	0	0	0	0	0
hazard_control_unit_in...	42	2	0	0	20	42	0	1	0	0
imem_inst (imem)	12	0	0	0	5	12	0	0	0	0
imm_extend_inst (im...	19	0	0	0	10	19	0	0	0	0
jump_branch_decisio...	1	0	0	0	1	1	0	0	0	0
memory_writeback_re...	0	129	0	0	61	0	0	0	0	0
mux2_ADDER_OPER...	16	0	0	0	11	16	0	0	0	0
mux2_branch_adres...	32	0	0	0	10	32	0	0	0	0
mux2_pc_value_inst (...)	16	0	0	0	10	16	0	0	0	0
mux3_forwardA_regist...	32	0	0	0	28	32	0	0	0	0
mux3_forwardB_regist...	32	0	0	0	28	32	0	0	0	0
mux4_register_input_i...	32	0	0	0	24	32	0	0	0	0
pc_inst (pc)	0	32	0	0	10	0	0	0	0	0
reg_file_inst (reg_file)	386	1024	160	0	329	386	0	0	0	0

Resource	Utilization	Available	Utilization %
LUT	2467	133800	1.84
LUTRAM	1024	46200	2.22
FF	1623	267600	0.61
IO	34	500	6.80



4. References

- 1- DDCA by “Sarah L Harris” and “David Money Harris”
- 2- The RISC-V Instruction Set Manual Volume I: Unprivileged ISA